



Vibe Coding: The Art of Natural Language Programming

A Comprehensive Guide to Modern AI-Assisted Development

Author: Daze Lu, Haijian Shao

Institute: University of Nevada, Las Vegas & Jiangsu University of Science and Technology

Date: Jan. 16, 2026

Version: I

Contents

I THE PARADIGM SHIFT	1
Chapter 1 Large Language Models	2
1.1 The Rise and Evolution of Large Language Models	2
1.2 Key Components of Large Language Models	2
1.3 Opportunities Enabled by Large Language Models	3
1.4 Major LLM Vendors and Their Models	4
Chapter 2 Vibe Coding	7
2.1 The Evolution of Programming	7
2.2 What Is Vibe Coding?	9
2.3 The Significance of Vibe Coding	10
2.4 Current Limitations	11
Chapter 3 AI Technology Stack	12
3.1 Prompt	12
3.2 Agent	12
3.3 MCP (Model Context Protocol)	13
3.4 Skills	14
3.5 Git Usage in AI-Assisted Development	15
II THE CORE CRAFT	17
Chapter 4 Requirements Analysis	18
4.1 Understanding Requirements in Vibe Coding	18
4.2 Requirements Gathering Techniques	18
4.3 Analyzing and Prioritizing Requirements	19
Chapter 5 Design	20
5.1 Design Thinking in Vibe Coding	20
5.2 Leveraging AI in Requirements and Design	20
5.3 Common Pitfalls and Best Practices	21
Chapter 6 Implementation	22
Chapter 7 Testing & Development & Maintenance	23
III ENGINEERING PRACTICES	24
7.1 cursor setup	33

Chapter 8 AI Agent System Building	37
Chapter 9 Web Development	38
9.1 Blog System	38
9.2 Control Code Version and Save on Github	40
9.3 make your project publicly accessible	42
Chapter 10 vibe coding in data science	44
Chapter 11 Mobile Development	45
Chapter 12 Desktop Development	46
Chapter 13 Game Development	47
Bibliography	48

Part I

THE PARADIGM SHIFT

Chapter 1 Large Language Models

1.1 The Rise and Evolution of Large Language Models

Large language models (LLMs) are neural networks trained on massive text corpora to predict the next word in a sequence. The field began with simple statistical models such as n-grams, progressed to recurrent neural networks, and later to transformer architectures introduced by Vaswani et al. in 2017 [1]. Transformers enable efficient parallel processing and capture long-range dependencies, leading to breakthroughs like BERT, GPT-2, and GPT-3. Each generation increased model size, data volume, and capability, moving from basic language understanding to generating coherent, context-aware text across many domains.

1.2 Key Components of Large Language Models

Transformer architecture. The transformer architecture (Figure 1.1) is the foundation of modern large language models. It consists of an encoder and decoder, both built from stacked layers. Each layer contains two main components: multi-head self-attention and feed-forward networks. The self-attention mechanism allows the model to weigh the importance of different words in a sentence when processing each word, enabling it to capture relationships regardless of distance. Multi-head attention runs several attention operations in parallel, allowing the model to focus on different aspects simultaneously. Positional encodings are added to input embeddings to preserve word order information, since attention itself is permutation-invariant. Residual connections and layer normalization stabilize training and enable deep architectures. This design allows Transformers to process entire sequences in parallel rather than sequentially, making training much more efficient than earlier recurrent models.

Attention Mechanism. The core of the Transformer is the scaled dot-product attention. For an input sequence, the model creates three vectors for each position: query (Q), key (K), and value (V). The attention score between a query and all keys is computed by taking their dot products, scaling by the square root of the key dimension d_k , and applying a softmax function to obtain weights. These weights are then used to compute a weighted sum of the values. Mathematically, the attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.1)$$

Multi-head attention extends this by running h parallel attention layers, each with different learned linear projections. The outputs are concatenated and projected again:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (1.2)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. This allows the model to attend to information from different representation subspaces.

Encoder and Decoder. The encoder consists of N identical layers, each with two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network. A residual connection and layer normalization are applied around each sub-layer. The output of each sub-layer can be written as $\text{LayerNorm}(x + \text{Sublayer}(x))$. The decoder also has N identical layers, but with three sub-layers: masked multi-head self-attention (to prevent positions from attending to future positions), encoder-decoder attention (queries come from the previous decoder layer, keys and values from the encoder output), and a feed-forward network. The final decoder output is projected to the vocabulary size and passed through a softmax to produce the probability distribution

over the next token.

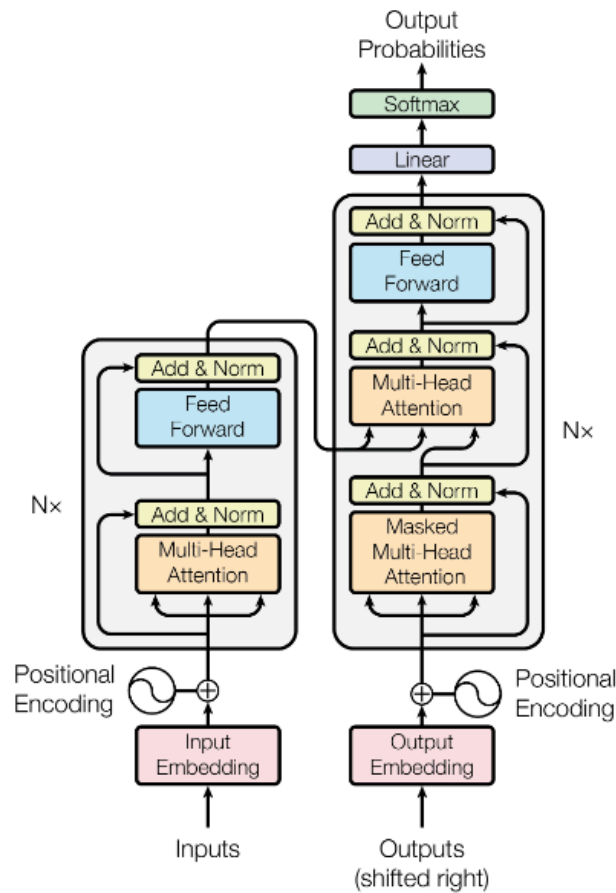


Figure 1.1: Transformer Architecture

1.3 Opportunities Enabled by Large Language Models

The rise of LLMs has opened a wide range of practical applications across multiple domains, transforming how we work with information and interact with technology.

Document Processing. LLMs have revolutionized document workflows. They can summarize lengthy research papers, legal contracts, or technical reports into concise overviews. Tools like ChatGPT and Claude can extract key information from documents, answer questions about their content, and generate structured outputs such as meeting notes or executive summaries. They also assist in writing tasks, from drafting emails and reports to creating marketing copy and academic papers. Additionally, LLMs power advanced search capabilities, understanding user intent and providing contextual answers rather than simple keyword matches.

Video and Audio Applications. In multimedia processing, LLMs work alongside speech recognition and computer vision systems. They can transcribe and summarize video content, generate subtitles and captions, and create detailed descriptions of visual scenes for accessibility. Models like Whisper (for speech-to-text) combined with LLMs enable automatic podcast transcription, meeting summaries, and video content indexing. They can also generate scripts for videos, suggest edits based on content analysis, and help in content moderation by detecting inappropriate material.

Translation and Multilingual Tasks. Modern LLMs have achieved near-human quality in machine translation across dozens of languages. Beyond simple translation, they preserve context, idiomatic expressions, and

cultural nuances. Applications include real-time conversation translation, website localization, cross-lingual information retrieval, and multilingual customer support. Models like GPT-4 and Google’s Gemini can translate technical documentation, literary works, and even code comments, making knowledge accessible across language barriers.

Programming and Software Development. LLMs have become invaluable coding assistants. GitHub Copilot, powered by OpenAI Codex, suggests code completions and entire functions based on context and comments. ChatGPT and Claude help developers debug code, explain complex algorithms, convert code between programming languages, and generate unit tests. They can review code for potential bugs, suggest optimizations, and help with documentation. Tools like Cursor and Replit’s Ghostwriter integrate LLMs directly into development environments, enabling natural language programming where developers describe what they want and the AI generates working code. This “vibe coding” paradigm represents a fundamental shift in how software is created.

1.4 Major LLM Vendors and Their Models

The large language model landscape is dominated by several key players, each bringing different strengths, philosophies, and capabilities. Understanding the major vendors and their flagship models helps developers choose the right tool for their needs and stay informed about the rapidly evolving AI ecosystem.

OpenAI. OpenAI pioneered the modern LLM era with the GPT (Generative Pre-trained Transformer) series. **GPT-3** (2020) demonstrated unprecedented scale with 175 billion parameters and showcased few-shot learning—the ability to perform new tasks with minimal examples. **GPT-4** (2023) marked a significant leap in reasoning, multimodal understanding (text and images), and reliability. GPT-4 excels at complex problem-solving, creative writing, coding, and nuanced conversation. OpenAI’s models power *ChatGPT*, one of the most widely used conversational AI systems, as well as enterprise APIs for developers. OpenAI also released specialized models like **Codex** (optimized for code generation, powering GitHub Copilot) and **DALL·E** (image generation). OpenAI’s strength lies in continuous refinement, extensive safety research (RLHF—Reinforcement Learning from Human Feedback), and a robust API ecosystem. As of early 2025, **GPT-4 Turbo** and the anticipated **GPT-5** continue to push boundaries in performance and context length.

Meta (Llama). Meta’s **LLaMA** (Large Language Model Meta AI) series represents a significant open-source alternative to proprietary models. **LLaMA 2** (2023), released under a permissive license for research and commercial use, offers models ranging from 7 billion to 70 billion parameters. LLaMA models emphasize efficiency and accessibility, enabling researchers and developers to run powerful LLMs on more modest hardware. **LLaMA 3** (2024) improved reasoning, multilingual capabilities, and instruction following, competing closely with GPT-4 on many benchmarks. Meta’s open approach has fostered a vibrant ecosystem of fine-tuned variants (e.g., Vicuna, Alpaca) and community-driven innovation. LLaMA’s strength is democratization: by open-sourcing strong base models, Meta enables experimentation, customization, and deployment without dependency on proprietary APIs. However, open models require users to handle hosting, fine-tuning, and safety considerations independently.

Anthropic (Claude). Anthropic, founded by former OpenAI researchers, focuses on AI safety and alignment. Their **Claude** series emphasizes harmlessness, helpfulness, and honesty. **Claude 2** (2023) introduced extended context windows (100K tokens initially, later 200K), enabling processing of entire books or codebases in a single prompt. **Claude 3** family (2024) includes **Opus** (most capable), **Sonnet** (balanced performance and speed), and **Haiku** (fast and cost-effective). Claude models excel at nuanced understanding, structured

reasoning, and long-document analysis. They are known for polite, thoughtful responses and strong refusal mechanisms to avoid harmful outputs. Claude is accessible via Anthropic's API and integrated into tools like Notion and Slack. Anthropic's Constitutional AI approach, which trains models to follow ethical principles, distinguishes Claude in safety-critical applications.

Google (Gemini, PaLM, Bard). Google has developed several LLM lines. **PaLM 2** (Pathways Language Model, 2023) underpins many Google services, offering strong multilingual capabilities and reasoning. **Gemini** (2023–2024) is Google's flagship multimodal model family, designed to natively handle text, images, audio, and video. **Gemini Ultra** competes directly with GPT-4, while **Gemini Pro** serves as the balanced mid-tier option, and **Gemini Nano** targets on-device applications (e.g., smartphones). Gemini powers *Bard*, Google's conversational AI, and integrates deeply into Google Workspace (Docs, Gmail, Sheets). Google's advantage lies in integration with its vast ecosystem (Search, YouTube, Android) and infrastructure (TPUs for efficient training and inference). Google's models also emphasize grounding—connecting LLM outputs to real-time web information and verified sources to reduce hallucination.

X.AI (Grok). Founded by Elon Musk, X.AI developed **Grok**, designed to be witty, less filtered, and more willing to engage with controversial topics compared to competitors. Grok is integrated into X (formerly Twitter), providing users with conversational AI that can access real-time posts and trends. **Grok-1** (2023) and subsequent versions emphasize humor, directness, and up-to-date information. X.AI also open-sourced Grok-1's base model, contributing to the open-source LLM ecosystem. Grok's positioning is as a less constrained, more opinionated assistant, appealing to users seeking unfiltered perspectives. Its integration with X's platform gives it access to a unique real-time data stream, differentiating it from models trained solely on static datasets.

Alibaba (Qwen). Alibaba's **Qwen** (Tongyi Qianwen) series focuses on multilingual and Chinese-language excellence. **Qwen-7B** and **Qwen-72B** compete with Western models on Chinese benchmarks while maintaining strong English performance. Qwen models support multiple modalities (text, vision) and emphasize practical applications in e-commerce, customer service, and enterprise tools within Alibaba's ecosystem. Alibaba also released **CodeQwen**, optimized for code generation, and open-sourced several Qwen variants to encourage adoption in China and globally. Qwen's strength lies in its deep understanding of Chinese language nuances, cultural context, and integration with Alibaba Cloud services, making it a top choice for Asia-Pacific enterprises.

Mistral AI (Mixtral). Based in France, Mistral AI champions open-source, efficient LLMs. **Mistral 7B** (2023) outperformed many larger proprietary models on a per-parameter basis, showcasing the power of efficient architecture and training. **Mixtral 8x7B** (2023) uses a sparse mixture-of-experts (MoE) architecture, activating only a subset of parameters per token, achieving high performance with lower computational cost. Mixtral models excel in reasoning, coding, and multilingual tasks while being deployable on consumer-grade hardware. Mistral AI's commitment to open-source (Apache 2.0 license) and European AI sovereignty has garnered strong community support. Mistral's models are ideal for developers seeking state-of-the-art performance without vendor lock-in.

Zhipu AI (GLM). Zhipu AI, a leading Chinese AI research lab, developed the **GLM** (General Language Model) series. **ChatGLM** and its successors (e.g., **ChatGLM2**, **ChatGLM3**) balance Chinese and English language capabilities, offering bilingual conversational AI. GLM models emphasize efficiency and can run on limited hardware, making them accessible for academic research and small-scale deployment. Zhipu AI also released **CogView** for text-to-image generation and integrates GLM into enterprise solutions. GLM's open-source versions have fostered innovation in the Chinese AI community, and its performance on Chinese NLP benchmarks rivals or exceeds many international competitors.

Trends and Takeaways. The LLM landscape is characterized by rapid innovation, with models doubling

in capability every 12–18 months. Key trends include: (1) *Multimodality*—models increasingly handle text, images, audio, and video natively; (2) *Longer context windows*—from 4K tokens (GPT-3) to 200K+ (Claude 3, GPT-4 Turbo); (3) *Open-source momentum*—LLaMA, Mistral, and others challenge proprietary dominance; (4) *Specialization*—models optimized for code (Codex, CodeQwen), reasoning (o1-preview), or safety (Claude); (5) *Efficiency*—mixture-of-experts (Mixtral) and quantization techniques reduce costs.

For Vibe Coding practitioners, choosing a model depends on task requirements: OpenAI and Anthropic excel in general-purpose reasoning and safety, LLaMA and Mistral offer customization and cost control, Google integrates deeply with productivity tools, and regional models (Qwen, GLM) provide linguistic expertise. Understanding this landscape enables developers to leverage the right model for each project phase, balancing capability, cost, privacy, and ecosystem fit.

Chapter 2 Vibe Coding

2.1 The Evolution of Programming

Programming has evolved through distinct stages, each representing a shift in the relationship between humans and computers.

In the earliest days, programmers wrote **assembly language**, working directly with machine instructions and memory addresses. This required humans to think like machines—tracking registers, managing memory manually, and expressing logic in terms of low-level operations. While powerful, assembly language demanded that humans adapt completely to the machine’s way of thinking.

The introduction of **high-level languages** like FORTRAN, C, and later Python and Java, reversed this relationship. These languages allowed programmers to express ideas in forms closer to human reasoning—using variables with meaningful names, control structures like loops and conditionals, and functions that encapsulate behavior. The compiler or interpreter translated these high-level abstractions into machine code, making the computer adapt to human thinking.

Object-oriented programming and abstraction further advanced this evolution by introducing concepts like classes, inheritance, and encapsulation. These paradigms helped manage complexity in large systems by organizing code around real-world concepts and hiding implementation details. Programmers could build complex systems by composing simpler, reusable components.

As software systems grew larger, **frameworks and engineering practices** emerged to enable large-scale collaboration. Version control systems, testing frameworks, design patterns, and architectural principles like microservices allowed teams to work together effectively. The focus shifted from individual coding skill to systematic engineering methodology.

Now, we enter **the AI era**, where the paradigm shifts once again. With large language models, programming transitions from “writing code” to “designing evolvable systems.” Developers no longer need to type every line of code; instead, they describe what they want, and AI generates implementations. The human role evolves toward architecture, constraints, verification, and iterative refinement.

2.1.1 Mainstream Programming Languages: Strengths and Use Cases

As programming evolved, different languages emerged to address specific needs, paradigms, and domains. Understanding the strengths and typical use cases of mainstream languages helps developers choose the right tool for each task—a decision that remains crucial even in the AI era, where language choice shapes what AI can generate effectively.

Python. Python has become one of the most popular languages due to its simplicity, readability, and extensive ecosystem. Its intuitive syntax makes it ideal for beginners, while powerful libraries enable advanced applications. Python dominates in *data science and machine learning* (NumPy, pandas, scikit-learn, TensorFlow, PyTorch), *web development* (Django, Flask), *automation and scripting*, and *scientific computing*. Python’s interpreted nature and dynamic typing enable rapid prototyping, though this comes at the cost of runtime performance compared to compiled languages. Python excels when development speed and expressiveness matter more than raw execution speed.

JavaScript / TypeScript. JavaScript is the language of the web, running in every browser and enabling

interactive web applications. With Node.js, JavaScript extended to server-side development, enabling full-stack development in a single language. TypeScript, a statically-typed superset of JavaScript, adds type safety and better tooling without sacrificing JavaScript's flexibility. Use cases include *frontend development* (React, Vue, Angular), *backend services* (Node.js, Express), *mobile apps* (React Native), and *desktop applications* (Electron). JavaScript's event-driven, asynchronous model handles I/O-bound tasks efficiently, making it well-suited for real-time applications, APIs, and user interfaces.

Java. Java remains a cornerstone of enterprise software due to its stability, performance, portability ("write once, run anywhere"), and mature ecosystem. Java's static typing, strong object-oriented design, and extensive standard library support large-scale, long-lived systems. Common use cases include *enterprise applications* (Spring framework), *Android mobile development*, *distributed systems* (microservices with Spring Boot), and *big data processing* (Hadoop, Spark). Java's JVM (Java Virtual Machine) ecosystem also supports other languages like Kotlin and Scala, which inherit Java's strengths while offering modern syntax and features.

C / C++. C and C++ provide low-level control over hardware and memory, enabling maximum performance and efficiency. C is procedural and minimal, ideal for systems where simplicity and predictability are critical. C++ extends C with object-oriented and generic programming features, supporting complex abstractions without sacrificing performance. Use cases include *operating systems* (Linux kernel in C), *embedded systems and IoT devices*, *game engines* (Unreal Engine in C++), *high-performance computing* (scientific simulations), and *real-time systems* where latency and resource constraints are critical. C and C++ require manual memory management, increasing complexity but providing unmatched control.

C#. Developed by Microsoft, C# is a modern, statically-typed language designed for the .NET ecosystem. It combines the performance and type safety of Java with modern language features like *async/await*, LINQ (Language Integrated Query), and rich library support. C# is widely used for *Windows desktop applications* (WPF, WinForms), *game development* (Unity engine), *web development* (ASP.NET Core), and *enterprise software*. The .NET runtime's cross-platform support (via .NET Core) has expanded C#'s reach beyond Windows to Linux and macOS.

Go (Golang). Go, created by Google, emphasizes simplicity, concurrency, and performance. Its goroutines and channels make concurrent programming straightforward, while its compiled nature ensures fast execution. Go's minimalist design and strong standard library support building efficient, scalable backend systems. Primary use cases include *cloud services and microservices* (Docker, Kubernetes written in Go), *APIs and web servers*, *DevOps tools*, and *distributed systems*. Go's fast compilation and deployment make it ideal for modern cloud-native development.

Rust. Rust combines the performance and control of C and C++ with memory safety guarantees enforced at compile time, eliminating entire classes of bugs (null pointer dereferences, data races). Rust's ownership system ensures safe concurrency without garbage collection. Use cases include *systems programming* (operating systems, file systems), *WebAssembly modules* for high-performance web applications, *blockchain and cryptography* (where security is paramount), and *embedded systems*. Rust's steep learning curve is offset by its reliability and growing ecosystem.

Swift. Swift is Apple's modern language for iOS, macOS, watchOS, and tvOS development. It replaces Objective-C with a safer, more concise syntax while maintaining interoperability with existing codebases. Swift emphasizes safety (optional types prevent null errors), performance (compiled and optimized), and expressiveness. It is the primary choice for *Apple ecosystem app development*, though efforts like Swift on Linux aim to broaden its reach.

Kotlin. Kotlin, developed by JetBrains, is a modern alternative to Java that runs on the JVM. It offers

concise syntax, null safety, functional programming features, and seamless Java interoperability. Kotlin is now Google’s preferred language for *Android development*, but also supports *server-side development* (Ktor framework), *multiplatform projects* (sharing code between Android, iOS, and web), and *enterprise applications*. Kotlin reduces boilerplate while maintaining Java’s ecosystem and performance.

Ruby. Ruby emphasizes developer happiness and productivity through elegant, readable syntax. Ruby on Rails, a powerful web framework, popularized convention-over-configuration and rapid prototyping. Ruby is commonly used for *web development* (Rails, Sinatra), *scripting and automation*, and *DevOps tools*. While Ruby’s interpreted nature makes it slower than compiled languages, its expressiveness and mature ecosystem accelerate development for web applications and startups.

PHP. PHP powers a significant portion of the web, including platforms like WordPress, Drupal, and Laravel. It is designed specifically for *server-side web development*, with built-in features for handling HTTP requests, sessions, and databases. Despite criticisms of legacy design choices, modern PHP (version 7+) has improved performance and introduced features like type declarations and JIT compilation. PHP remains relevant for *content management systems*, *e-commerce platforms*, and *web applications* where quick deployment and extensive hosting support are priorities.

SQL. While not a general-purpose programming language, SQL (Structured Query Language) is essential for interacting with relational databases. SQL enables *data querying, manipulation, and analysis* across systems like MySQL, PostgreSQL, Oracle, and SQL Server. Understanding SQL is critical for backend development, data engineering, and analytics. Variants like PL/SQL and T-SQL extend SQL with procedural capabilities for complex database logic.

Shell Scripting (Bash, Zsh). Shell scripts automate tasks in Unix-like systems (Linux, macOS). They excel at *system administration, build automation, CI/CD pipelines*, and *environment setup*. While limited in expressiveness compared to Python or Ruby, shell scripts integrate seamlessly with system utilities and are ubiquitous in DevOps workflows.

R. R is a specialized language for *statistical computing and data visualization*. It offers extensive libraries for statistical modeling, hypothesis testing, and plotting (ggplot2). R is widely used in academia, research, and industries requiring rigorous statistical analysis, such as bioinformatics and finance. While Python increasingly competes in data science, R remains preferred for specialized statistical tasks.

In the AI era, language choice matters differently than before. AI models excel at generating code in well-documented languages with large training corpora (Python, JavaScript, Java). However, the fundamental strengths and trade-offs of each language remain: Python for rapid prototyping and data science, Rust for safety-critical systems, Go for scalable services, C++ for performance. Understanding these distinctions allows developers to guide AI effectively, choosing languages that align with project requirements and leveraging AI to accelerate implementation within those constraints.

2.2 What Is Vibe Coding?

Vibe Coding is a term that captures the essence of AI-assisted software development: programming by expressing intent and iteratively refining results, rather than writing code line by line. The term emphasizes the shift from precise syntactic control to high-level direction—developers communicate the “vibe” or desired outcome, and AI tools generate implementations that match this vision.

The concept emerged organically within developer communities as large language models became capable coding assistants. While the exact origin is difficult to pinpoint, the term gained traction around 2023–2024

as tools like GitHub Copilot, ChatGPT, and Claude demonstrated the ability to generate substantial code from natural language descriptions. Developers began describing their workflow not as “writing code” but as “vibing” with AI—providing context, describing goals, reviewing outputs, and iterating until the result feels right.

Current Technologies. Several categories of tools enable Vibe Coding today:

Code completion and generation. Tools like **GitHub Copilot** (powered by OpenAI Codex) and **Amazon CodeWhisperer** integrate directly into IDEs, suggesting code completions, entire functions, or even multi-file implementations based on context and comments. They operate inline, making AI-generated code feel like an extension of the developer’s own typing.

Conversational coding assistants. General-purpose LLMs like **ChatGPT**, **Claude**, and **Google Gemini** allow developers to describe problems in natural language and receive code solutions, explanations, debugging help, and refactoring suggestions. These tools excel at exploratory programming, prototyping, and learning new technologies.

Integrated development environments. Next-generation IDEs like **Cursor** and **Windsurf** are built around AI-first workflows. They combine code editing, chat interfaces, codebase understanding, and multi-file refactoring in a unified experience. Developers can ask questions about their codebase, request changes across multiple files, and iterate on design decisions conversationally.

Specialized AI agents. Tools like **Devin**, **Replit Agent**, and **v0 by Vercel** go beyond code generation to perform end-to-end tasks. They can scaffold entire projects, implement features across frontend and backend, run tests, debug failures, and deploy applications. These agents represent the frontier of autonomous software development.

Prompt engineering frameworks. As Vibe Coding matures, structured approaches to prompting emerge. Techniques like chain-of-thought prompting, few-shot examples, and systematic context management improve reliability and reproducibility. Frameworks and methodologies for prompt-driven development are evolving rapidly.

While these technologies are powerful, they are also nascent and evolving. The challenge is to harness their potential within a disciplined engineering framework—hence the need for a Vibe Coding methodology.

2.3 The Significance of Vibe Coding

Vibe Coding represents a transformation comparable to the historical transition from assembly to high-level languages. Just as high-level languages freed programmers from managing registers and memory addresses, AI-assisted coding frees them from writing boilerplate, routine logic, and even complex algorithms character by character.

The fundamental shift is this: **humans no longer control how code is written, but rather what kinds of implementations are allowed.** Instead of specifying every detail of syntax and control flow, developers define the system’s architecture, constraints, interfaces, and desired behavior. The AI generates code that satisfies these specifications, and the developer’s role becomes validation, refinement, and ensuring the result meets quality standards.

This shift changes what constitutes programming skill. Syntactic correctness—remembering function signatures, avoiding off-by-one errors, properly closing brackets—becomes less critical. Instead, **structural and constraint correctness** takes center stage. Can you design an architecture that scales? Can you write tests that catch edge cases? Can you specify clear interfaces and invariants? Can you reason about system behavior and verify correctness?

In this new paradigm, **engineering capability becomes the key determinant of upper-bound productivity**. A developer with strong architectural thinking, testing discipline, and domain knowledge can leverage AI to build far more than they could by writing code manually. Conversely, a developer without these skills will generate code that compiles but lacks coherence, maintainability, and reliability. The bottleneck shifts from typing speed to design quality.

2.4 Current Limitations

Despite its promise, current AI-assisted coding workflows face significant challenges that must be addressed for Vibe Coding to reach its full potential.

Fragility of prompt-driven workflows. Many developers interact with AI through ad-hoc prompts, requesting code snippets without systematic context. Small changes in phrasing can produce wildly different results. Without a structured methodology, prompt-driven development becomes unpredictable and unreliable, especially as projects grow in complexity.

Risk of AI-generated code that appears correct but is fundamentally wrong. Large language models can generate syntactically valid code that passes superficial inspection but contains subtle bugs, security vulnerabilities, or logic errors. Without rigorous testing and verification, these issues may not surface until production. Developers must remain skeptical and validate AI outputs thoroughly.

Unmaintainability caused by missing architecture. When AI generates code without a unifying architectural vision, the result is often a collection of disconnected components that work initially but become difficult to extend or modify. Lack of clear separation of concerns, inconsistent patterns, and tightly coupled dependencies turn codebases into maintenance nightmares.

Irreproducibility due to lack of testing. Code generated in one session may not be regenerable in another, especially if the context or prompt changes slightly. Without comprehensive tests that define expected behavior, there is no reliable way to verify that regenerated or modified code maintains correctness.

The goal of this book is to present **a Vibe Coding methodology that remains stable over time**. By combining systematic prompt engineering, architectural discipline, automated testing, and iterative refinement, we aim to make AI-assisted development not just fast, but reliable, maintainable, and reproducible. This methodology treats AI as a powerful tool within a rigorous engineering framework, rather than a replacement for engineering judgment.

Chapter 3 AI Technology Stack

Modern Vibe Coding relies on a layered technology stack that extends beyond raw language models. Understanding these components—prompts, agents, MCP (Model Context Protocol), and skills—is essential for building effective AI-assisted development workflows. Each layer addresses different aspects of the development process, from low-level interaction to high-level capability composition.

3.1 Prompt

The Scope and Limits of Prompt-Based Control. Prompts are the primary interface for instructing language models. A prompt can be as simple as a question or as complex as a multi-page specification with examples, constraints, and context. Prompts excel at conveying intent, providing examples, and guiding model behavior for specific tasks. They enable rapid iteration: adjust the prompt, regenerate output, and refine until the result is satisfactory.

However, prompts have fundamental limitations. They are ephemeral—once the conversation ends, the prompt’s effect disappears unless manually reapplied. They are brittle—small changes in wording can produce drastically different outputs. They lack composability—complex systems cannot be built by concatenating prompts; the cognitive load and context window constraints become overwhelming. Prompts are excellent for one-off tasks, exploratory coding, and prototyping, but insufficient for systematic, large-scale development.

Differences Between Prompts and Structural Constraints. A prompt is a suggestion; a structural constraint is a rule. Prompts guide the model toward desired behavior, but they do not enforce it. For example, a prompt might say “write clean, well-documented code,” but the model may still produce poorly commented or convoluted code. In contrast, structural constraints—such as type systems, linters, test suites, and architectural guardrails—enforce correctness mechanically. A failing test is a hard constraint; a prompt asking for correct code is a soft guideline.

Effective Vibe Coding combines both: prompts provide high-level direction and context, while structural constraints (tests, schemas, interfaces) ensure the generated code meets non-negotiable requirements. Prompts describe the “what” and “why”; constraints verify the “how” and “whether.”

Why Prompts Cannot Handle Full Engineering Design Responsibilities. Engineering involves long-term thinking: modularity, extensibility, maintainability, performance, security, and compatibility. These concerns cannot be fully captured in a single prompt. A prompt cannot encode the entire architectural vision of a system, the historical context of design decisions, or the subtle trade-offs between competing requirements.

Moreover, prompts do not persist across sessions. If a developer relies solely on prompting without documentation, tests, or structured project files, the system’s design exists only in the developer’s head and scattered conversation history. This makes onboarding new team members, revisiting old code, or regenerating components nearly impossible. Engineering requires stable, explicit artifacts—design documents, interfaces, tests, configuration files—that outlive any single prompt.

3.2 Agent

Overview of Agent Frameworks. Agents extend language models with the ability to take actions, use tools, and orchestrate multi-step workflows. Popular agent frameworks include **AutoGen** (Microsoft), **CrewAI**,

LangGraph (LangChain ecosystem), and **Anthropic’s Claude with tool use**. These frameworks allow agents to call APIs, run code, search documentation, query databases, and interact with external systems.

An agent typically operates in a loop: receive a task, plan steps, execute actions using tools, observe results, and iterate until the task is complete. For example, an agent tasked with “implement a REST API for user management” might: (1) generate code, (2) run tests, (3) read error messages, (4) fix bugs, (5) verify functionality, and (6) report completion. This autonomous behavior significantly reduces manual intervention.

Strengths: Workflow Orchestration and Task Automation. Agents excel at automating repetitive, multi-step workflows. They can scaffold projects, generate boilerplate, run build scripts, execute tests, parse logs, and retry failed steps. Agents handle tasks that require iteration and feedback—for instance, debugging a failing test suite by running tests, reading stack traces, modifying code, and re-testing until success.

Agents also enable delegation. A developer can assign high-level tasks to an agent (“set up CI/CD pipeline,” “migrate database schema”) and focus on higher-level design while the agent handles implementation details. This accelerates development and reduces cognitive load.

Limitations: Agents Cannot Replace Architectural or Engineering Judgment. Despite their capabilities, agents have critical limitations. They struggle with ambiguous or open-ended problems that require human judgment. Agents lack deep understanding of business context, user needs, and long-term strategic goals. They can implement a feature, but they cannot decide whether the feature is worth building or how it fits into the product vision.

Agents can also make poor architectural decisions. Without explicit guidance, an agent might choose a quick, hacky solution over a clean, maintainable design. They may introduce tight coupling, ignore performance implications, or violate security best practices. Agents optimize for task completion, not system quality.

Furthermore, agents operate within the constraints of their tools and context. They cannot invent novel algorithms, rethink fundamental assumptions, or challenge flawed requirements. They execute plans, but they do not create vision. Effective use of agents requires human oversight: set clear goals, provide architectural constraints, review outputs, and intervene when the agent’s approach diverges from best practices.

3.3 MCP (Model Context Protocol)

What Is MCP? The Model Context Protocol (MCP) is an open standard developed by Anthropic to enable seamless integration between AI models and external data sources, tools, and services. MCP defines a uniform way for language models to access context—files, databases, APIs, documentation, and live system state—without requiring custom integrations for each tool.

Before MCP, connecting an AI to external resources required ad-hoc implementations: custom API wrappers, bespoke data formatters, and tool-specific prompts. MCP standardizes this process. It specifies how resources (e.g., files, database tables) and tools (e.g., search, execute command) are exposed to models, how models request access, and how results are returned.

MCP operates through “servers” that provide resources and tools. For example, an MCP server for a Git repository might expose resources like file contents and commit history, along with tools for creating branches or committing changes. The AI client (e.g., Claude, Cursor) connects to MCP servers and uses the exposed capabilities seamlessly.

What Can MCP Do? MCP enables several powerful workflows for Vibe Coding:

Codebase-aware assistance. By connecting to an MCP server that exposes the entire codebase, an AI can understand project structure, read existing code, and generate changes that fit the context. This reduces

hallucination and improves relevance.

Live system integration. MCP servers can expose real-time data—database schemas, API endpoints, application logs, or running service metrics. The AI can query this data to inform its suggestions, debug issues, or generate code that integrates correctly with the live environment.

Tool orchestration. MCP allows AI to invoke tools like compilers, test runners, linters, and deployment scripts. The AI can compile code, run tests, interpret errors, and iterate—all through MCP-exposed tools.

Documentation and knowledge retrieval. MCP servers can index documentation, internal wikis, or research papers. The AI can search this knowledge base to answer questions or incorporate best practices into generated code.

MCP's standardization means that once a developer sets up MCP servers for their tools and resources, any MCP-compatible AI client gains access. This creates a composable, extensible ecosystem for AI-assisted development.

3.4 Skills

3.4.1 what skill is

Skills as Stable, Reusable Capability Units. In Vibe Coding, a **skill** is a well-defined, reusable capability that an AI can invoke to perform a specific task. Unlike prompts, which are ad-hoc and ephemeral, skills are structured and persistent. A skill encapsulates instructions, examples, context, and sometimes executable code or tool configurations needed to accomplish a task reliably.

For example, a "code review" skill might include: (1) instructions for what to check (style, logic, security), (2) examples of good and bad code, (3) a checklist of common issues, and (4) links to style guides or linting rules. When invoked, the AI uses this skill to perform consistent, high-quality code reviews.

Skills make AI behavior predictable and reproducible. Instead of re-writing detailed prompts for recurring tasks, a developer invokes a named skill, ensuring consistent results across sessions and team members.

Interface-Driven, Testable, and Composable Skill Design. Well-designed skills follow software engineering principles. They have clear inputs and outputs—interfaces that define what the skill expects and returns. This makes skills composable: one skill can call another, building complex workflows from simple, tested components.

Skills should be testable. A "unit test generation" skill should reliably produce syntactically correct tests that cover specified cases. Testing skills ensures they remain effective as models and tools evolve. Skills can be versioned, reviewed, and improved systematically, just like code.

Composability allows developers to build personal toolkits or domain-specific libraries of skills. For instance, a data scientist might have skills for "data cleaning," "exploratory analysis," "model training," and "visualization." These skills can be chained to automate entire analysis pipelines.

Relationship Between Skills, Personal Toolkits, and Research Kernels. Skills aggregate into **personal toolkits**—collections of capabilities tailored to a developer's workflow, domain, and preferences. A web developer's toolkit might include skills for "API scaffolding," "frontend component generation," "database migration," and "deployment automation." Over time, these toolkits become force multipliers, encoding expertise and best practices.

Research kernels extend this idea to collaborative, domain-specific knowledge bases. A **research kernel** is a curated collection of skills, documentation, schemas, and tools for a particular domain (e.g., bioinformatics,

robotics, game development). Teams can share and evolve research kernels, creating a collective intelligence layer that enhances AI-assisted development.

Skills bridge the gap between raw AI capability and disciplined engineering practice. They bring structure, reusability, and quality control to Vibe Coding, transforming ad-hoc prompting into systematic methodology.

3.4.2 Add skill in Cursor

3.5 Git Usage in AI-Assisted Development

In traditional software development, Git serves as version control—a way to track code changes, collaborate with teammates, and maintain project history. In Vibe Coding, Git’s role expands significantly. It becomes not just a record of code, but a record of the **human–AI collaboration process itself**: the experiments attempted, the design decisions made, the hypotheses tested, and the iterations refined.

Git as the Factual Record of Human–AI Collaboration. When AI generates code, the commit history tells the story of how that code came to be. Each commit captures a snapshot of the codebase at a moment when the developer decided “this is worth preserving.” This is crucial in AI-assisted development because code evolution is often non-linear. Developers try multiple approaches, compare alternatives, backtrack when a direction proves unfruitful, and iterate rapidly based on AI suggestions.

Without Git, this process becomes invisible. There is no record of what was tried, what failed, or why certain decisions were made. The final codebase appears as if it materialized fully formed, hiding the reasoning and exploration that led to it. With Git, every meaningful state is preserved. Future developers (including your future self) can understand not just *what* the code does, but *why it exists in its current form*.

A Timeline of Experiments, Hypotheses, and Design Decisions. In Vibe Coding, commits should capture intent, not just changes. A good commit message in an AI-assisted workflow reads like a lab notebook entry: “Experiment: tried caching strategy A, performance improved by 20%” or “Hypothesis: extracting this logic into a helper function will improve testability.” The commit itself is the evidence; the message is the interpretation.

This approach transforms Git history into a navigable timeline of your development process. When a feature behaves unexpectedly, you can trace back through commits to find when and why a particular design choice was made. When evaluating alternative implementations, you can compare branches side-by-side, each representing a different hypothesis tested systematically.

Branches, in this context, are not just for feature development—they are for **experimentation**. Before asking an AI to refactor a critical module, create a branch. Try the refactoring, run tests, evaluate the result. If it works, merge it. If it introduces problems, discard the branch and try a different approach. Every uncertain attempt should be isolated in its own branch, making it easy to compare alternatives and choose the best solution.

The Foundation for Rollback, Comparison, and Auditability. AI-generated code is not always correct on the first try. Sometimes an AI’s suggestion looks plausible but introduces subtle bugs, performance regressions, or architectural inconsistencies. Git provides a safety net: if a change causes problems, you can roll back to the last known-good state instantly.

More importantly, Git enables **comparison**. You can diff two implementations, inspecting exactly what changed. This is invaluable when debugging AI-generated code: instead of asking “what’s wrong with this entire file,” you ask “what changed in this specific commit that broke functionality?” The problem space shrinks dramatically.

Git also provides **auditability**. In professional settings, it may be necessary to demonstrate that code meets certain standards, passes security reviews, or adheres to regulatory requirements. A well-maintained Git history serves as evidence: "this security vulnerability was identified in commit X, fixed in commit Y, and verified by tests in commit Z." For AI-assisted development, this auditability is especially important, as stakeholders may question whether AI-generated code is trustworthy. A transparent history showing iterative refinement, testing, and validation builds confidence.

In summary, Git is not optional in Vibe Coding—it is foundational. It transforms the ephemeral, conversational nature of AI-assisted development into a durable, inspectable, and improvable engineering artifact. Developers should commit early, commit often, and write commit messages that capture the *why* behind every change. This discipline pays dividends in maintainability, collaboration, and long-term project health.

Part II

THE CORE CRAFT

Chapter 4 Requirements Analysis

Requirements analysis forms the foundation of successful software projects. In traditional development, this phase involves gathering user needs and defining system behavior before writing code. In Vibe Coding, requirements analysis remains critical, but its nature evolves: AI can accelerate exploration and validate feasibility, but human judgment determines what to build and why. This chapter explores how to conduct effective requirements analysis in the AI era.

4.1 Understanding Requirements in Vibe Coding

Requirements as Constraints, Not Specifications. In traditional development, requirements are often written as detailed specifications: "The system shall display a list of users sorted alphabetically with pagination of 20 items per page." AI-assisted development shifts the focus toward **constraints and outcomes**: "Users need to view a large list of users efficiently." The AI can propose implementations (sorting, pagination, infinite scroll, search filters), and the developer evaluates which best meets the actual need.

This does not mean requirements become vague. Instead, they become **testable outcomes and invariants**. Good requirements in Vibe Coding answer: What problem are we solving? Who benefits? What defines success? What must not happen (security, performance, compatibility constraints)? These high-level goals guide AI-generated solutions while leaving implementation flexibility.

User-Centered vs. Solution-Centered Requirements. A common pitfall is jumping to solutions before understanding problems. "We need a mobile app" is a solution-centered requirement. "Users need to access their account information on the go" is user-centered. Vibe Coding amplifies this risk: AI can quickly generate code for any stated solution, tempting developers to skip deep problem analysis. Resist this. Spend time understanding *why* a feature is needed, *who* will use it, and *what outcomes* define success. Document these insights before asking AI to generate designs.

Functional vs. Non-Functional Requirements. Functional requirements describe what the system does: "Users can log in with email and password." Non-functional requirements define how well it does it: "Login must complete in under 2 seconds," "Passwords must be stored with bcrypt hashing," "The system must support 10,000 concurrent users." In Vibe Coding, AI excels at implementing functional requirements but often ignores non-functional ones unless explicitly prompted. Always specify performance, security, scalability, and usability constraints upfront. These become the structural constraints that guide and validate AI outputs.

4.2 Requirements Gathering Techniques

Stakeholder Interviews and User Research. AI cannot replace direct interaction with users and stakeholders. Conduct interviews, surveys, and observation sessions to understand needs, pain points, and workflows. Ask open-ended questions: "What frustrates you about the current system?" "What would success look like?" "Walk me through a typical task." Record insights, not solutions. AI tools can help analyze interview transcripts, identify patterns, and summarize findings, but human empathy and intuition drive deep understanding.

Use Cases and User Stories. Document requirements as use cases (detailed step-by-step interactions) or user stories (concise statements of user goals). For example: "As a project manager, I want to generate weekly progress reports automatically so that I can save time and ensure consistency." Use cases and stories provide

context for AI: instead of asking AI to “build a reporting feature,” provide the full user story. AI can then propose implementations aligned with the user’s actual goal.

Prototyping and Iteration. In Vibe Coding, rapid prototyping becomes trivial. Use AI to generate mock-ups, wireframes, or even functional prototypes quickly. Show these to stakeholders, gather feedback, and iterate. This “build-to-think” approach clarifies ambiguous requirements faster than lengthy documentation. However, treat prototypes as disposable exploration, not production code. The goal is learning, not shipping the first draft.

Requirements Workshops and Brainstorming. Collaborative sessions with stakeholders, developers, and designers align understanding and surface hidden assumptions. Use techniques like mind mapping, affinity diagrams, and priority matrices to organize ideas. AI can facilitate by generating ideas, summarizing discussions (if transcribed), and suggesting missing considerations based on similar projects.

4.3 Analyzing and Prioritizing Requirements

Feasibility Analysis. Not all requirements are equally achievable. Assess technical feasibility (Can we build this with available technology?), economic feasibility (Is the cost justified?), and schedule feasibility (Can we deliver in time?). AI assists by quickly prototyping solutions to test feasibility. For example, if a requirement involves real-time video processing, ask AI to generate a proof-of-concept. If it runs too slowly, adjust the requirement or explore alternatives.

Prioritization: MoSCoW and Kano Models. Use frameworks like **MoSCoW** (Must have, Should have, Could have, Won’t have) to prioritize requirements. Must-haves are non-negotiable; Won’t-haves are out of scope. The **Kano model** categorizes requirements by user satisfaction: basic needs (expected), performance needs (more is better), and delighters (unexpected features that wow users). In Vibe Coding, focus AI effort on must-haves and delighters. AI can rapidly implement performance improvements, so these become lower priority.

Dependency Mapping. Identify dependencies between requirements: Which features must be built first? Which are independent? Create a dependency graph to guide implementation order. AI can help generate this graph from a list of requirements, highlighting blockers and suggesting optimal build sequences.

Risk Assessment. Every requirement carries risk: technical complexity, integration challenges, security vulnerabilities, or changing user needs. Assess risks early. For high-risk requirements, plan mitigation strategies: build prototypes, allocate extra time, or design fallback options. AI can surface risks by analyzing similar projects or code patterns, but human judgment evaluates severity and likelihood.

Chapter 5 Design

Design translates requirements into a blueprint for implementation. While requirements define what to build, design determines how to build it: the architecture, components, interfaces, and patterns that will shape the system. In Vibe Coding, design becomes even more critical because AI generates code based on the structures and constraints you define. A well-designed system enables AI to contribute effectively; a poorly designed system leads to chaos regardless of AI capability.

5.1 Design Thinking in Vibe Coding

Separation of Concerns and Modularity. Good design isolates functionality into independent, reusable components. Each module has a clear responsibility, well-defined interfaces, and minimal coupling. This principle is even more critical in Vibe Coding: modular designs make it easier to ask AI to implement or refactor one component without affecting others. Define module boundaries, interfaces, and contracts *before* asking AI to generate code.

Architectural Patterns. Choose architectural patterns that fit the problem: layered architecture (presentation, business logic, data access), microservices (independent, deployable services), event-driven architecture (decoupled components communicating via events), or Model-View-Controller (MVC) for UI applications. Document the chosen pattern and enforce it consistently. AI can generate code that follows a pattern if you describe it clearly, but without guidance, AI may produce inconsistent or anti-pattern code.

Interface Design: APIs, Schemas, and Contracts. Define interfaces early: API endpoints, function signatures, data schemas, and communication protocols. Interfaces are contracts that specify *what* components do, not *how*. Once interfaces are stable, multiple developers (human or AI) can work on different components in parallel without conflicts. Use tools like OpenAPI for REST APIs, Protocol Buffers for data serialization, or JSON Schema for data validation. AI excels at generating implementations that conform to predefined interfaces.

Design for Testability. Design systems that are easy to test. Avoid tight coupling, global state, and hidden dependencies. Use dependency injection to make components testable in isolation. Write interfaces and abstract classes to enable mocking. When asking AI to implement a feature, specify testability requirements: "Ensure this function is pure and has no side effects," or "Use dependency injection for database access." Testable designs enable rapid iteration and validation.

Design Documentation: Diagrams and ADRs. Document design decisions using **UML diagrams** (class diagrams, sequence diagrams, component diagrams) and **Architecture Decision Records (ADRs)**. UML diagrams visualize structure and behavior, making it easier for AI to understand context. ADRs document why a design choice was made, preventing future confusion. For example: "ADR-001: We chose PostgreSQL over MongoDB because our data is highly relational and ACID guarantees are critical." These artifacts guide AI and preserve design rationale.

5.2 Leveraging AI in Requirements and Design

Generating Design Alternatives. One of AI's strengths is exploration. Ask AI to propose multiple architectural approaches for a problem. For instance: "I need to build a notification system. Propose three different architectures: one using polling, one using WebSockets, and one using server-sent events. Compare their trade-

offs.” Review the proposals, ask clarifying questions, and choose the best fit. AI generates breadth; humans provide judgment.

Validating Designs Through Prototyping. Use AI to quickly build throwaway prototypes that test design assumptions. If you’re unsure whether a caching layer will improve performance, ask AI to generate a prototype with and without caching, then benchmark both. This evidence-based design reduces guesswork and surfaces issues early.

Refining Requirements Iteratively. Requirements evolve. As you learn more, adjust them. AI facilitates this: regenerate components with updated requirements, compare changes, and iterate. Maintain a living requirements document (e.g., a README or design doc) that reflects the current understanding. Update it as requirements change, and use it to context AI generations.

Detecting Gaps and Inconsistencies. AI can analyze requirements documents and flag missing details, contradictions, or ambiguous statements. For example: “This requirement says the system must support real-time updates, but there’s no mention of how conflicts are resolved when two users edit simultaneously.” Use AI as a requirements reviewer, but validate its findings critically.

5.3 Common Pitfalls and Best Practices

Pitfall: Skipping Design to “Just Start Coding.” AI’s speed tempts developers to skip design and dive into implementation. This leads to brittle, hard-to-maintain systems. Resist the urge. Invest time in upfront design. A well-designed system is easier to build, test, and extend—even with AI.

Pitfall: Over-Specifying Implementation Details. Conversely, don’t micromanage AI with overly detailed specifications. Specify *what* and *why*, not *how*. Let AI choose efficient implementations within your constraints. Over-specification reduces AI’s effectiveness and leads to rigid designs.

Best Practice: Iterate on Requirements and Design Together. Requirements inform design, and design informs requirements. As you design, you’ll discover requirements that are infeasible or incomplete. Embrace this feedback loop. Update requirements, adjust design, and repeat until both are coherent and achievable.

Best Practice: Use Visual Models. Diagrams, wireframes, and flowcharts clarify complex ideas faster than text. Use tools like Mermaid, PlantUML, or Figma to create visual models. Share these with AI to provide rich context for code generation.

Best Practice: Validate Early and Often. Don’t wait until implementation to validate requirements. Build prototypes, conduct user testing, and gather feedback continuously. Catching misunderstandings early saves time and prevents expensive rework.

Requirements analysis in Vibe Coding is not a phase you complete once and forget. It is an ongoing activity that guides AI, constrains solutions, and ensures the system meets real needs. The better your requirements, the more effectively AI can contribute to implementation. Invest in thorough requirements analysis, and AI becomes a force multiplier for delivering what users actually need.

Chapter 6 Implementation

Chapter 7 Testing & Development & Maintenance

Part III

ENGINEERING PRACTICES

before we get start

We use claude code as our AI coding tools, and use vscode as our code editor, and use git as our version control system, and use github as our code hosting platform. We need to set them up before we get start.

git setup

Let's firstly set up git, cause the cluade code needs git bash to run. steps:

- go to <https://git-scm.com/install/windows>
- download the git for windows installer, and run it, and follow the instructions to install it
- for the components selection part, check everything, and the others can be default, and click next

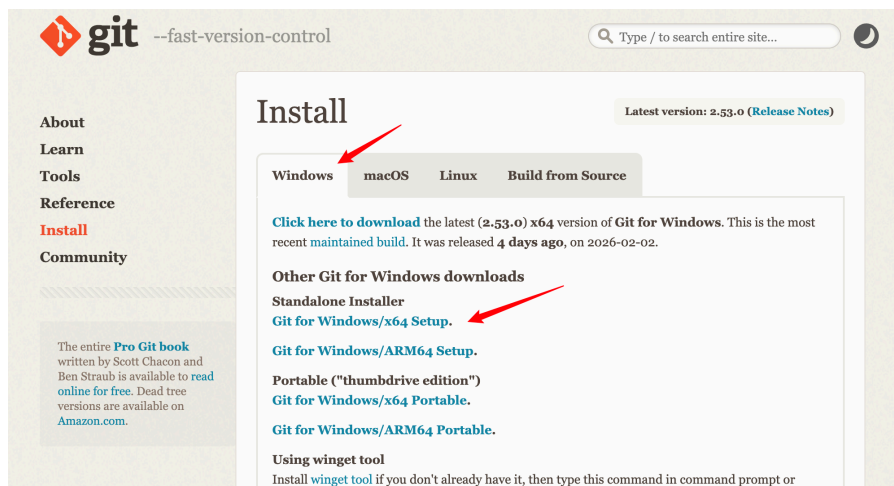


Figure 7.1: Installing Git on Windows

Select Components

Which components should be installed?

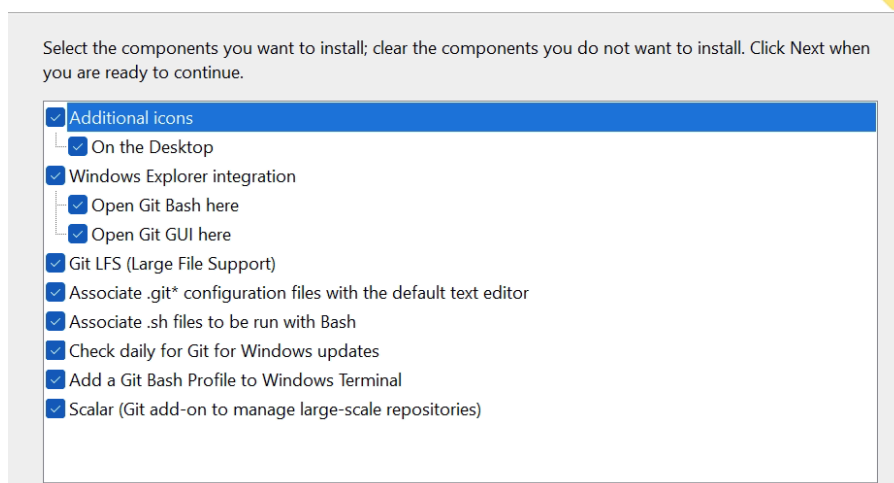


Figure 7.2: check everything in this step

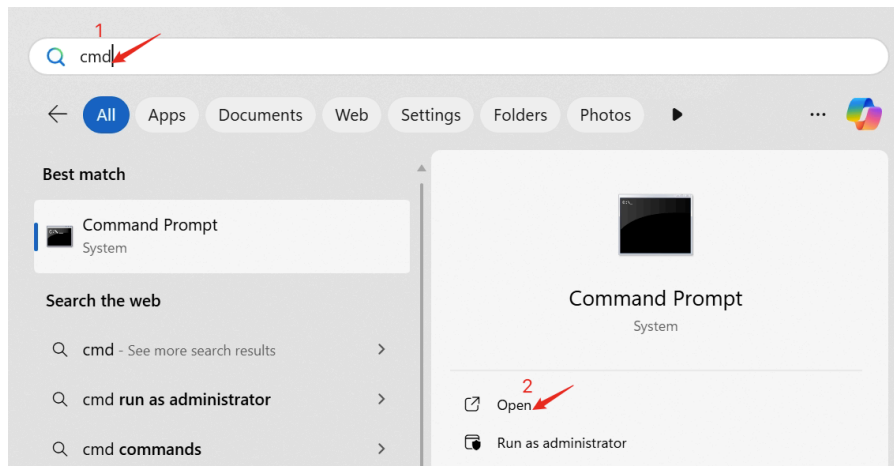


Figure 7.3: open the command prompt, and run "git --version" to check if git is installed successfully

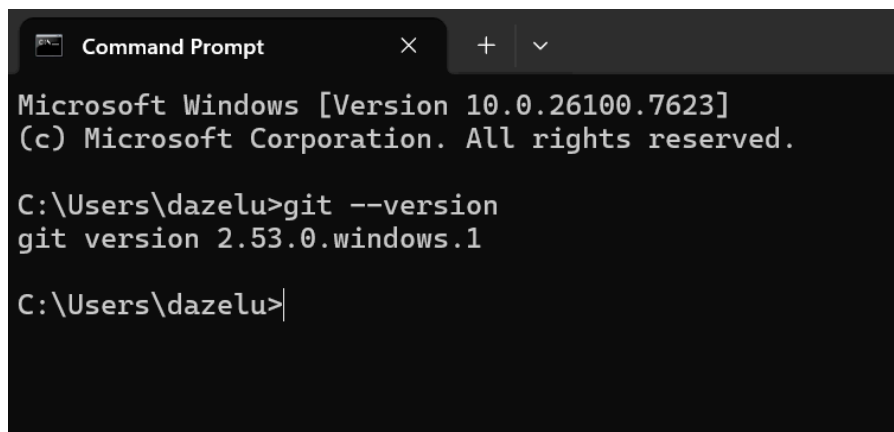


Figure 7.4: if you see this, it means you have successfully installed git

nodejs setup

To build up a web application, we need to set up nodejs environment, please go to <https://nodejs.org/en/download> and download the latest LTS version of nodejs, and run the installer, and follow the instructions to install it. If you have administration permission, it will be easy, if you don't have administration permission, we will install nodejs manually, in our case, we will do it manually.

steps:

- go to <https://nodejs.org/en/download>, and download the latest LTS version of nodejs for windows, and choose the binary zip version, not the installer version.
- after downloading, extract the zip file to a folder, for example, "D:\nodejs"
- add the nodejs folder to the PATH environment variable, so that we can run node and npm commands in the terminal, to do this, search "environment variable" in the windows search bar, and click "Edit the system environment variables", then click "Environment Variables", then find the "Path" variable in the "user variables" section, and click "Edit", then click "New", and add the path of the nodejs folder, for example, "D:\nodejs", then click "OK" to save the changes.
- open a new command prompt, and run "node -v" and "npm -v" to check if nodejs is installed successfully, if you see the version numbers, it means you have successfully installed nodejs.

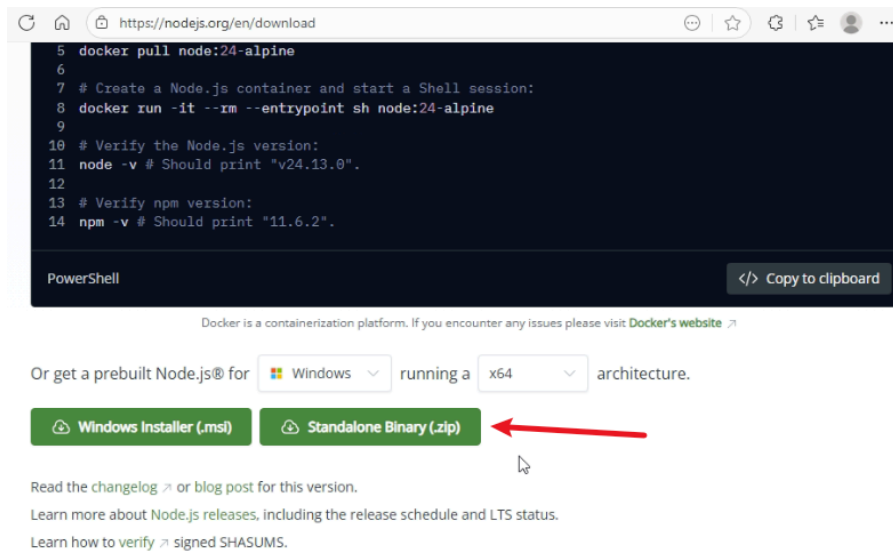


Figure 7.5: download nodejs binary zip file

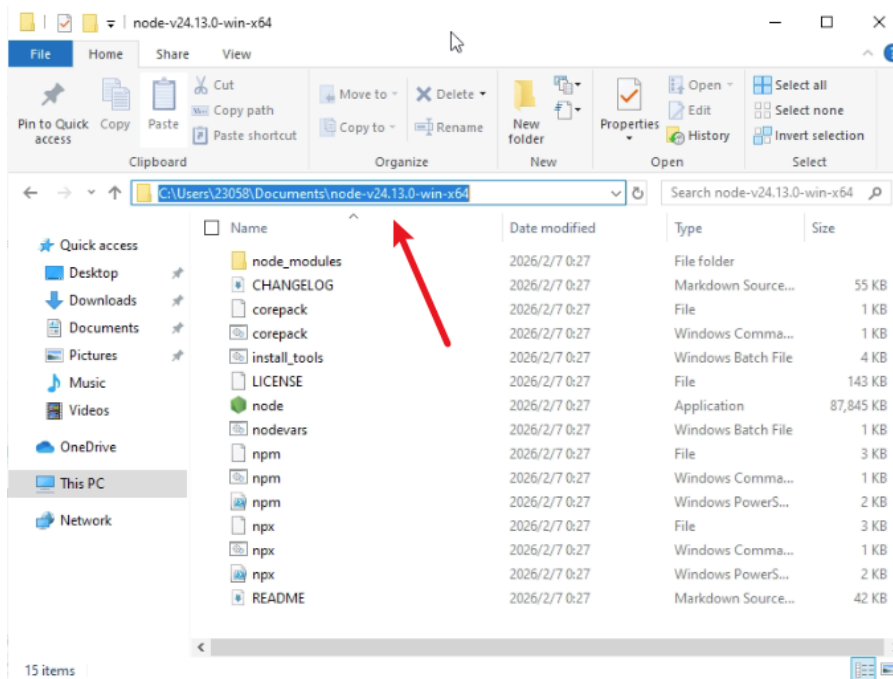


Figure 7.6: extract the nodejs zip file to a folder

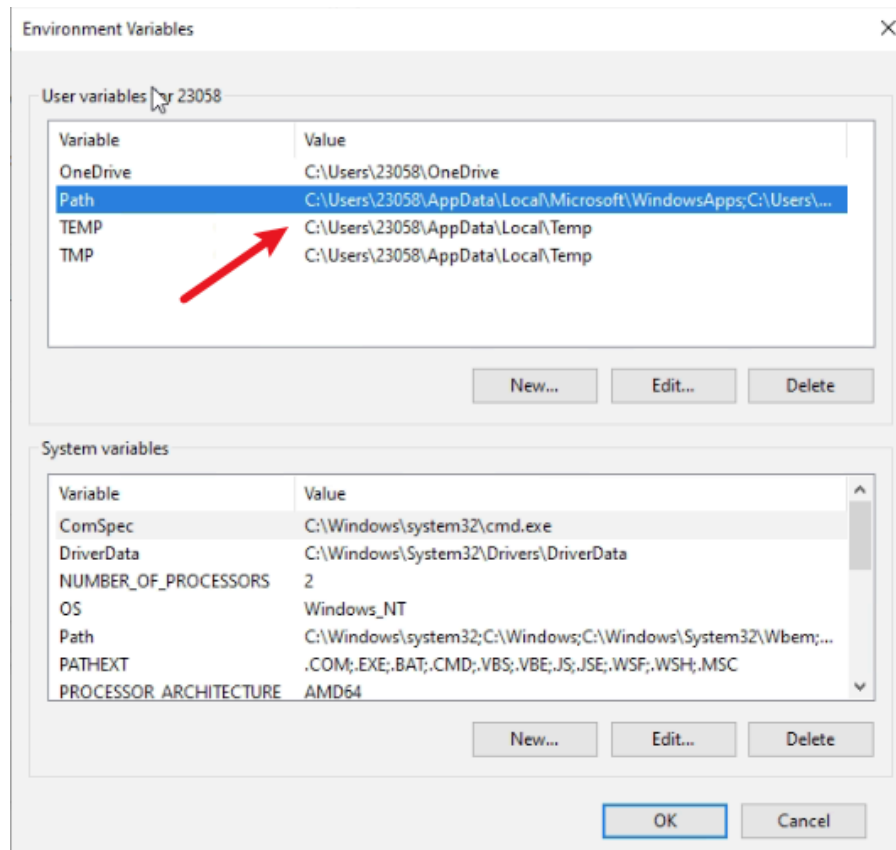


Figure 7.7: add nodejs to the PATH environment variable, click the edit button to add new path

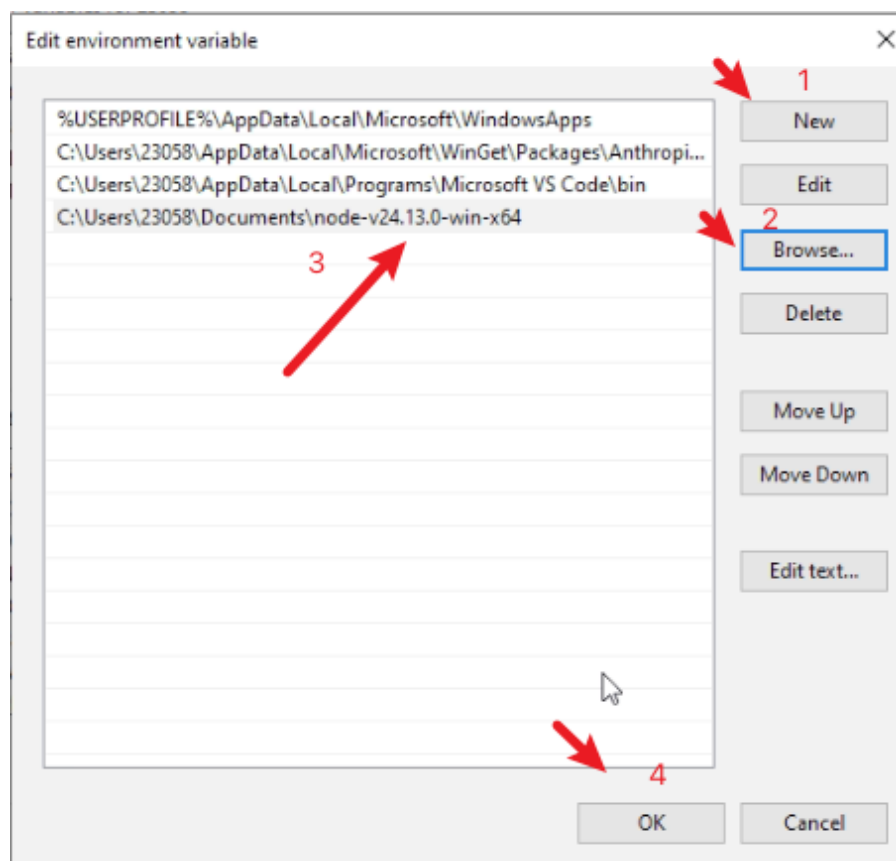
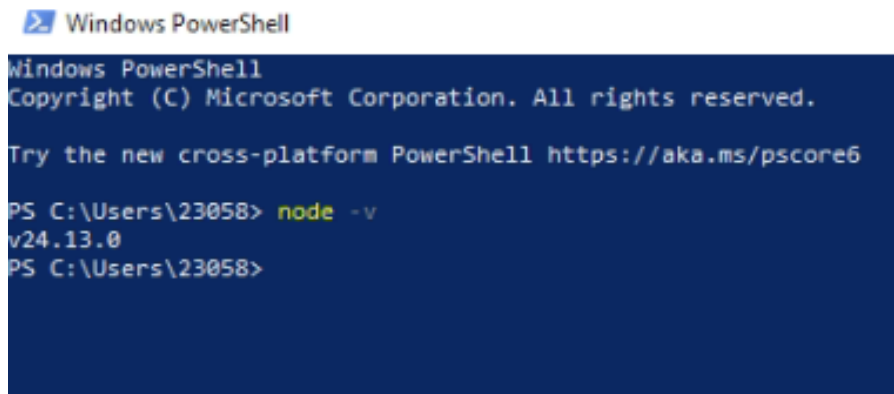


Figure 7.8: click new and add navigate to the nodejs folder to add it to the PATH environment variable



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

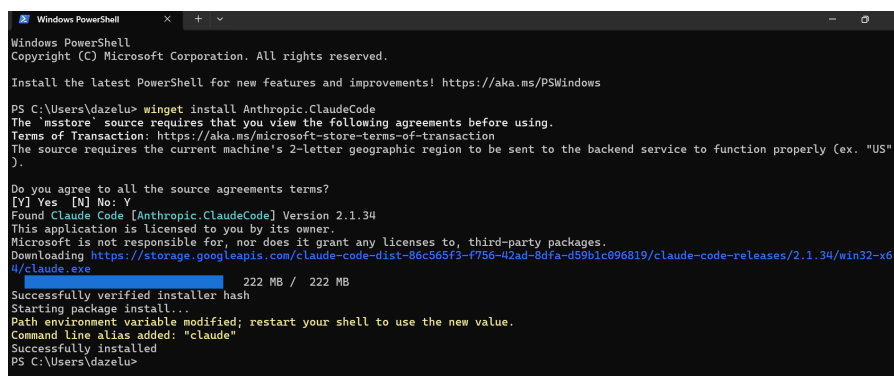
PS C:\Users\23058> node -v
v24.13.0
PS C:\Users\23058>
```

Figure 7.9: nodejs is successfully installed and added to PATH environment variable if you can see the version numbers after running "node -v" and "npm -v" in a new powershell or command prompt

claude code setup

Go to <https://code.claude.com/docs/en/overview>, you can find the installation instructions for different platforms. For example, for Windows, you can run the following command in PowerShell to install claude code, to find the powershell, in the top of the command prompt, click the down arrow, and select "Windows PowerShell", then run the command below:

```
# Windows (Winget):
winget install Anthropic.ClaudeCode
```

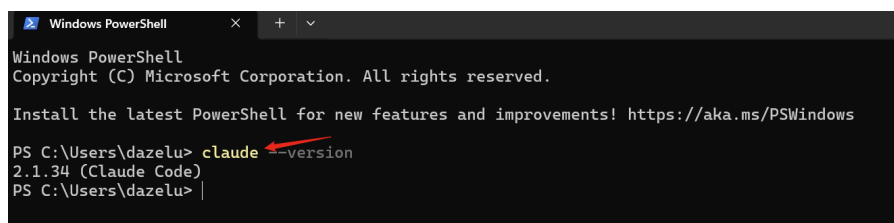


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\dazelu> winget install Anthropic.ClaudeCode
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").
Do you agree to all the source agreements terms?
[Y] Yes [N] No: Y
Found Claude Code [Anthropic.ClaudeCode] Version 2.1.34
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://storage.googleapis.com/claude-code-dist-86c565f3-f756-42ad-8dfa-d59b1c096819/claude-code-releases/2.1.34/win32-x64/claude.exe
222 MB / 222 MB
Successfully verified installer hash
Starting package install...
Path environment variable modified; restart your shell to use the new value.
Command line alias added: "claude"
Successfully installed
PS C:\Users\dazelu>
```

Figure 7.10: if you see this, it means you have successfully installed claude code



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\dazelu> claude --version
2.1.34 (Claude Code)
PS C:\Users\dazelu>
```

Figure 7.11: check the version in a new powershell, if you see the version number, it means you have successfully installed claude code

Next step, we need to setup the API key for claude code, we use z.ai API in claude code, because it's cheaper than the claude API. Go to <https://z.ai/>, and sign up for an account, then go to the API section, and create a new API key for claude, see this link for guide: <https://docs.z.ai/devpack/tool/claude>.

In this class, I'll provide my API-KEY for you to use, run these commands in the gitbash to set up the API key for claude code:

```
curl -O "https://cdn.bigmodel.cn/install/claude_code_zai_env.sh" && bash ./claude_code_zai_env.sh
```

then paste the following api-key

```
861e62ff738e40439f1e15e33f6e5a37.0pVFAhFjfMmhFrvG
```

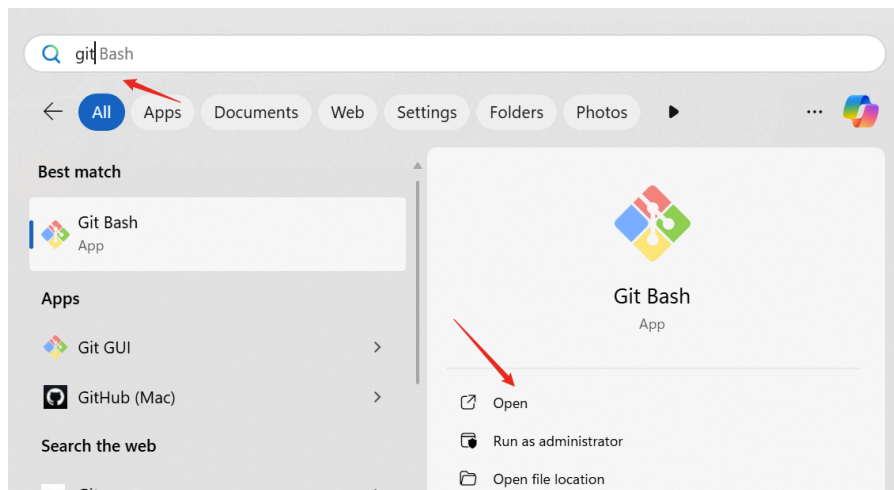


Figure 7.12: find gitbash

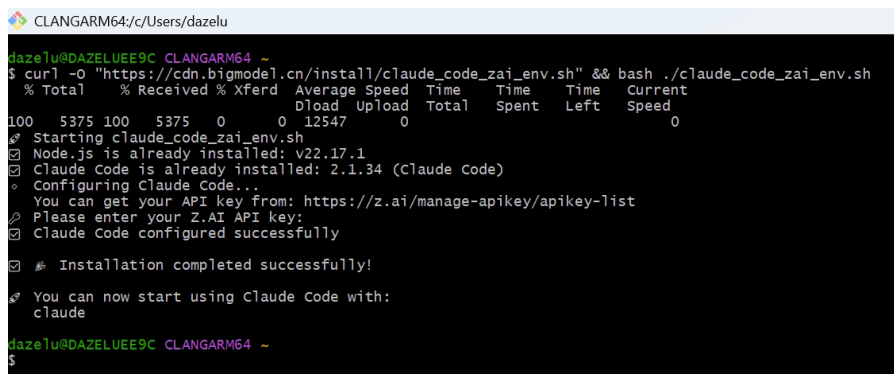


Figure 7.13: set up the API key for claude code in gitbash

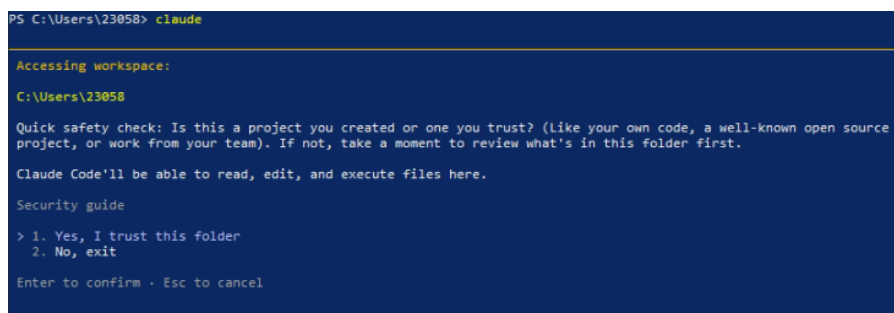


Figure 7.14: if you see this, it means you have successfully set up the API key for claude code, and you can start using it

By the way, if the claude code couldn't be setup, you can use cursor with this API-KEY, please follow this link for guidance: <https://docs.z.ai/devpack/tool/cursor>. And please download cursor from <https://cursor.com/download>.

Python Environment Setup

We use Miniconda to manage the Python environment, please go to <https://repo.anaconda.com/miniconda/> and download the latest version of Miniconda for Windows, remember check "add to path" option when installing it, after you have installed Miniconda, you can open the Anaconda Prompt, and run "conda -version" to check if conda is installed successfully, if you see the version number, it means you have successfully installed Miniconda.

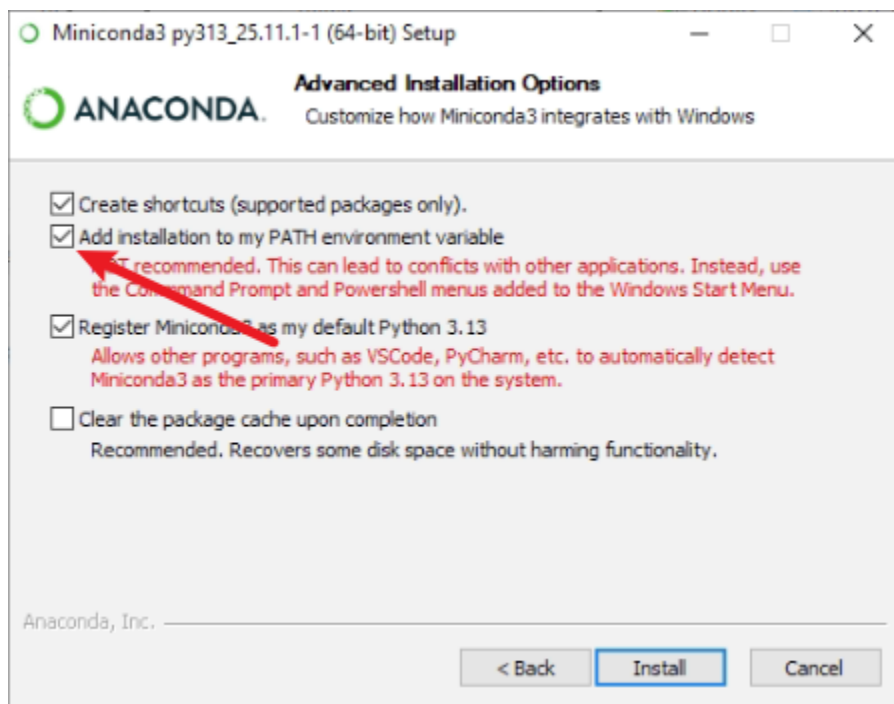


Figure 7.15: Add the conda to the PATH environment variable when installing Miniconda

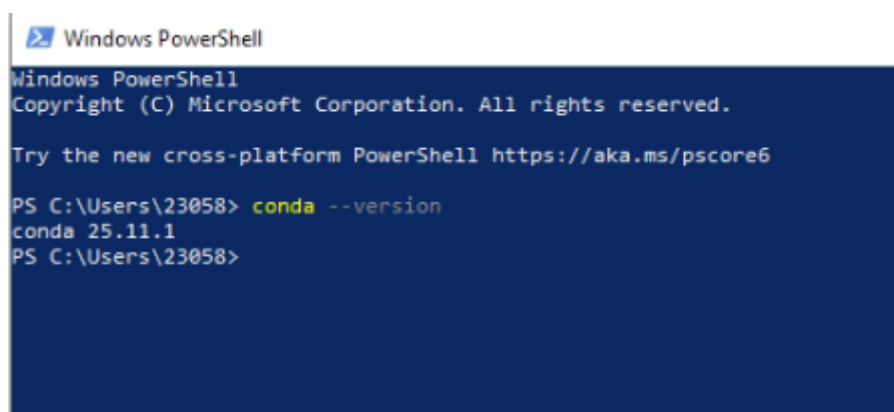


Figure 7.16: Check if install conda successfully by running "conda -version" in a new Anaconda Prompt, if you see the version number, it means you have successfully installed Miniconda

vscode setup

It's easy to set up vscode, please download from <https://code.visualstudio.com/>, you can just choose the default installation options. After you have installed vscode, you can open the folder of your project in vscode, and you can see the code in vscode, and you can also use the terminal in vscode to run git commands, and run the code.

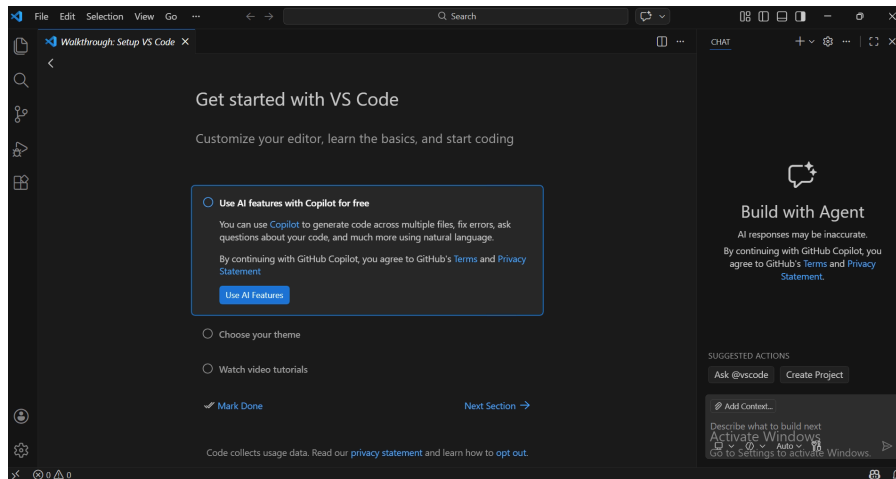


Figure 7.17: you can get page like this after you installing vscode and launching it for the first time

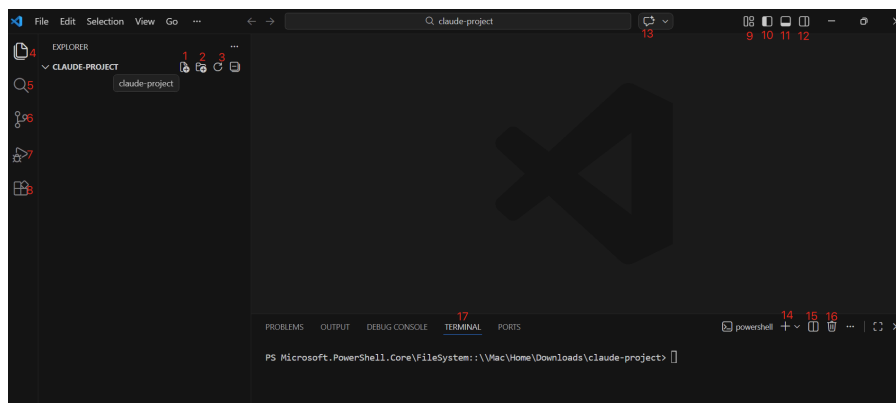


Figure 7.18: vscode introduction

Here is simple introduction for the vscode interface:

- **1 New File:** Create a new file in the current folder.
- **2 New Folder:** Create a new folder in the current folder.
- **3 Refresh:** Refresh the file explorer to show the latest changes.
- **4 Explorer Panel:** Shows the project files and folders.
- **5 Search:** Search text across the project.
- **6 Source Control:** Manage Git version control.
- **7 Run and Debug:** Run or debug the project.
- **8 Extensions:** Install and manage VS Code extensions.
- **9 Tools Bar:** Access various tools and commands.
- **10 Left Panel Controller:** Show/hide the left panel.
- **11 Bottom Panel Controller:** Show/hide the bottom panel.

- **12 Right Panel Controller:** Show/hide the right panel.
- **13 AI Calling:** Call AI assistant in VS Code.
- **14 New Terminal:** Create a new terminal.
- **15 Split Terminal:** Split the terminal window.
- **16 Close Terminal:** Close the current terminal.
- **17 Terminal Panel:** Integrated terminal for command execution.

github setup

Login hugub with your google account, we will use it to store your code.

7.1 cursor setup

Cursor is a premier AI-powered code editor and a highly effective alternative to Claude Code, offering several distinct advantages such as a seamless installation process and a much more intuitive interface for monitoring code diffs and logic changes. To get started, simply download the Windows version from cursor.com/download and complete the installation; you can then integrate the same API key you use for Claude Code to power the editor's features. For a detailed walkthrough on setting everything up, please refer to this comprehensive guide: docs.z.ai/devpack/tool/cursor.

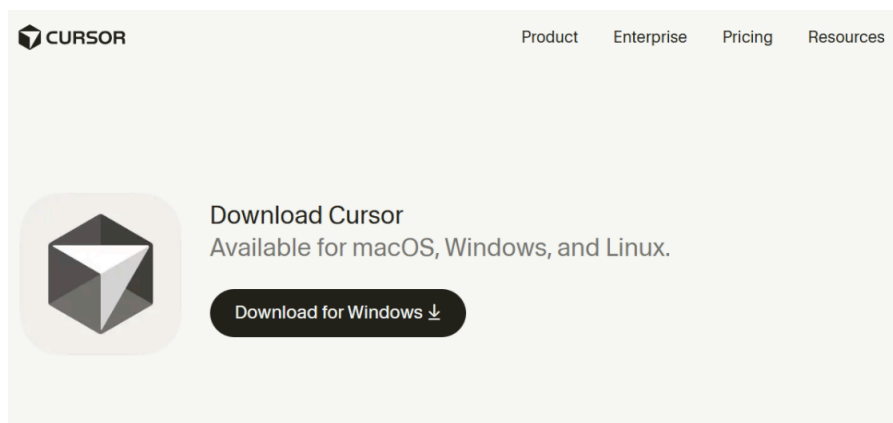


Figure 7.19: cursor download page

After downloading, double-click the installer and use the default options to complete the installation.

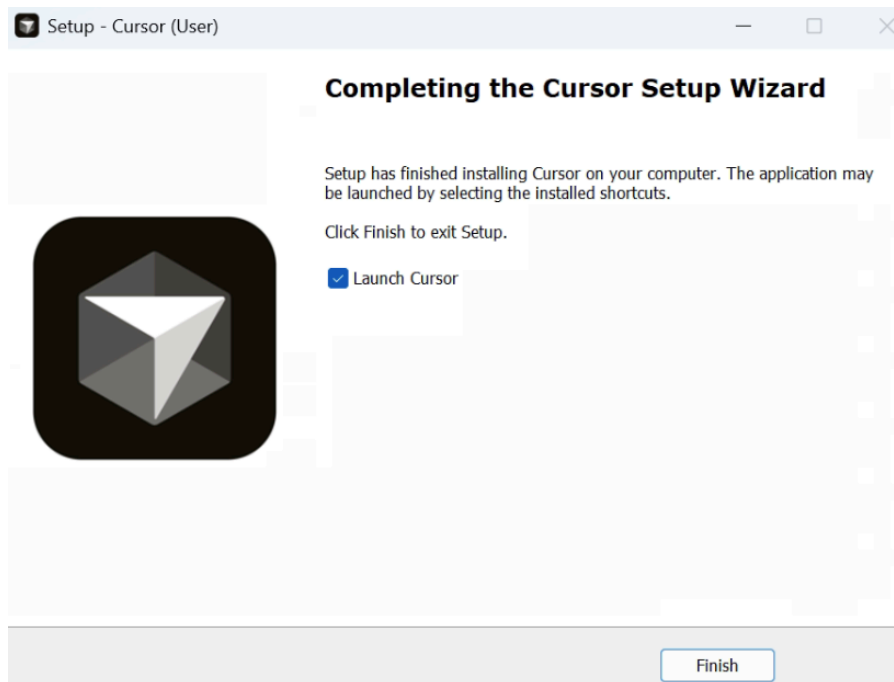


Figure 7.20: you will get the page like this after installing cursor successfully

Next, you need to login to use it, if you are under 18, please contact the instructor, he will assign you a account to use.

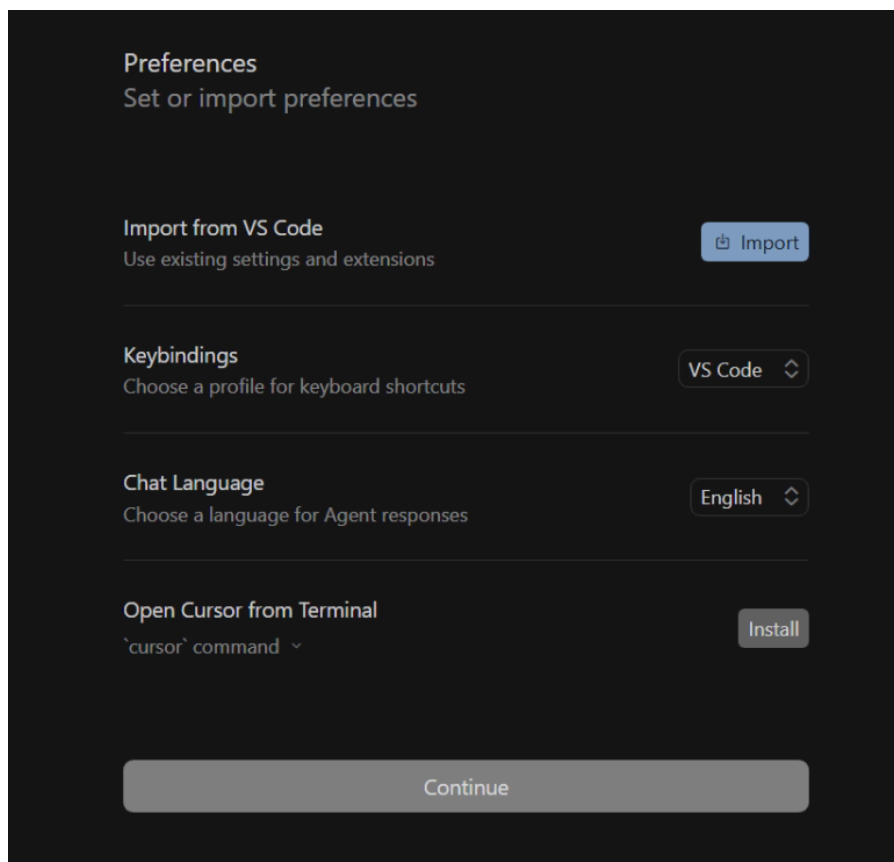


Figure 7.21: in this page, just click continue, use the default options.

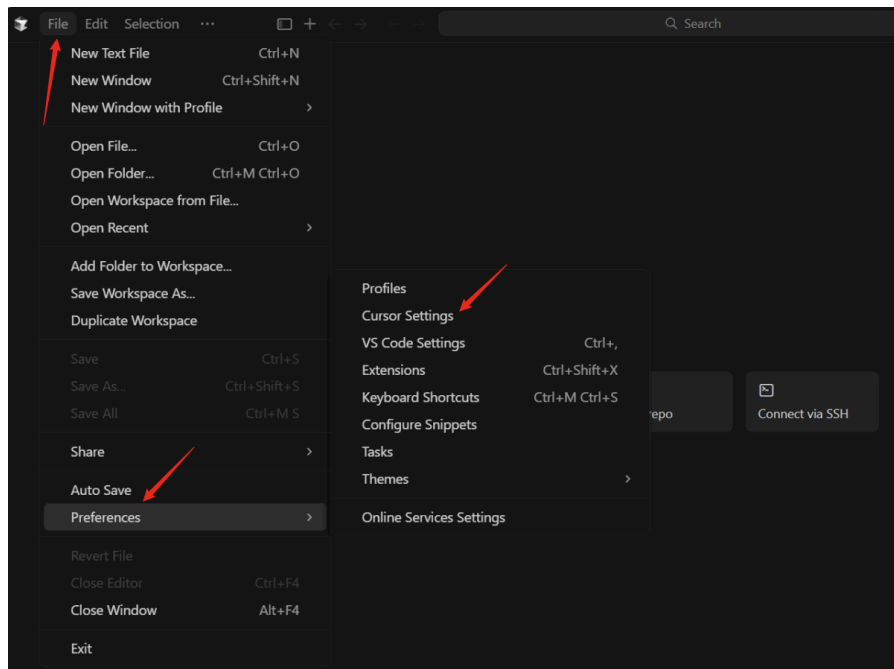


Figure 7.22: set the GLM API key for cursor.

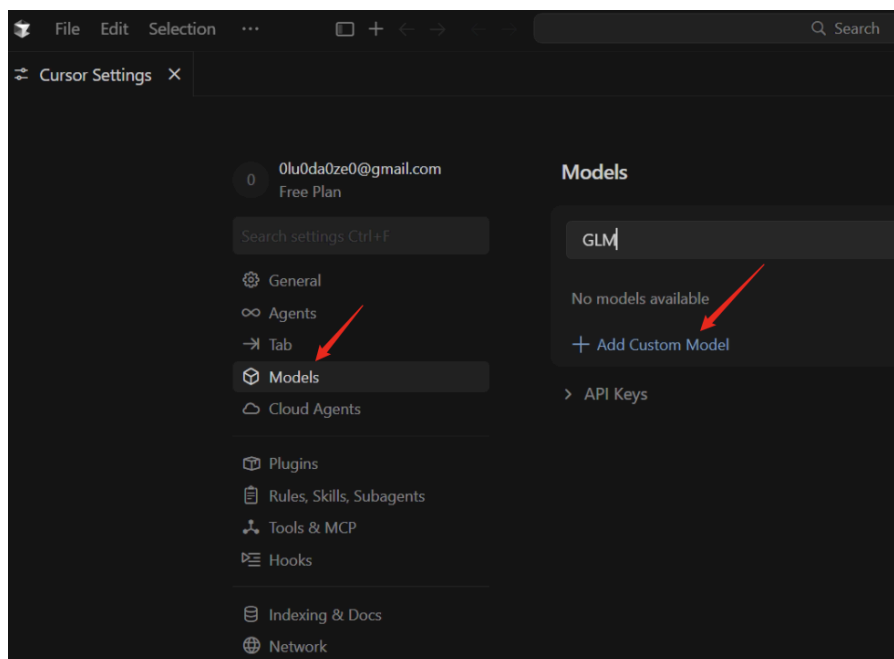


Figure 7.23: add models for cursor.

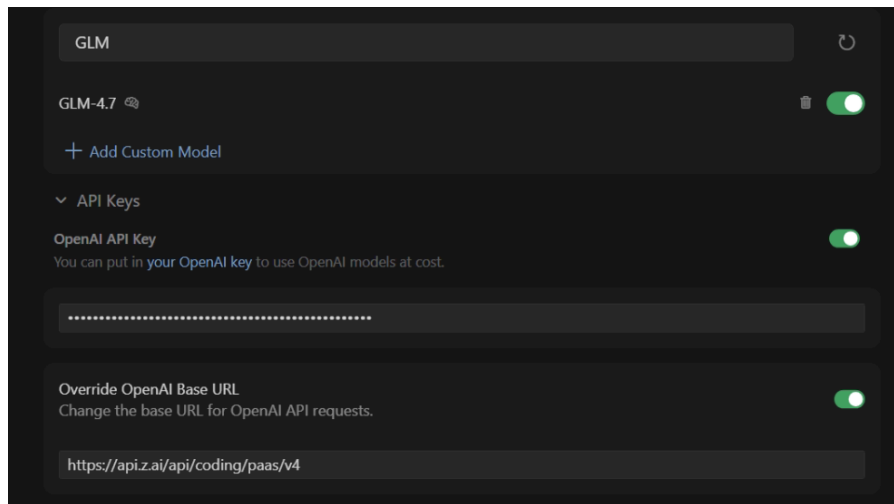


Figure 7.24: add glm-4.7 for cursor.

```
# base url
https://api.z.ai/api/coding/paas/v4

# api-key
861e62ff738e40439f1e15e33f6e5a37.0pVFAhFjfMmhFrvq
```

Chapter 8 AI Agent System Building

Chapter 9 Web Development

9.1 Blog System

System Overview In this project, we develop a blog service system. The backend is implemented using Python and Flask to provide RESTful APIs, with Python's built-in database (SQLite) used for data persistence. The system supports the following functionalities:

- User registration, login, and logout.
- Creation, editing, and publishing of new blog posts.
- Modification and deletion of already published blog posts.
- Support for multiple page themes, such as light mode and dark mode.
- Blog interaction features, including likes, dislikes and comments.
- Blog search functionality.

Finally, we will deploy the blog system through cloudflare, then people can access it publicly.

9.1.1 Product Requirements Document

Now we gonna use AI to give up a plan. Get into claude, and enable plan mode, then input the our system overview, then you just negotiate with the AI to give up a plan for the implementation of the blog system, you can ask the AI to break down the system into smaller components, and give up a plan for each component, and also give up a timeline for the implementation.

```
/model to try Opus 4.5
//plan
Enabled plan mode

System Overview In this project, we develop a blog service system. The backend is implemented using
Python and Flask to provide RESTful APIs, with Python's built-in database (SQLite) used for data persistence.
The system supports the following functionalities:
User registration, login, and logout.
Creation, editing, and publishing of new blog posts.
Modification and deletion of already published blog posts.
Support for multiple page themes, such as light mode and dark mode.
Blog interaction features, including likes, dislikes and comments.
Blog search functionality

I see you've entered plan mode and provided an overview of your blog service system. However, I don't see a specific task or feature you'd like me to help plan.
Let me ask you to clarify what you'd like to work on:

Task

1. Add a new feature
Implement functionality not currently described (e.g., tags, categories, user profiles, etc.)
2. Improve existing feature
Enhance something the system already does (e.g., better search, theme switching, etc.)
3. Fix a bug or issue
Debug or fix something that's not working correctly
4. Set up from scratch
Help design and plan the initial architecture for this blog system
5. Type something
6. Chat about this
7. Skip interview and plan immediately

Enter to select · r/4 to navigate · Esc to cancel
```

Figure 9.1: AI plan mode in claude

 **Note** Please let the AI generate the plan into a file, like 'task.md', you may need to modify it later.

```

Phase 8: Testing & Deployment
- Unit and integration tests
- Docker configuration
- CI/CD pipeline
- Deployment documentation
---
Critical Files
- backend/app/models/post.py - Post model with tag/category relationships
- backend/app/routes/posts.py - Post CRUD with filtering, search, auth
- backend/app/services/auth_service.py - JWT token generation/verification
- backend/app/extensions.py - Flask extensions (db, jwt, cors)
- frontend/src/components/posts/PostEditor.jsx - Toast UI Editor integration
- frontend/src/services/api.js - Axios with JWT interceptors
- frontend/src/contexts/AuthContext.jsx - Authentication state management
---
Verification
1. Backend: Run flask run and test API endpoints with Postman/Thunder Client
2. Frontend: Run npm run dev and verify full user flow (register → login → create post → add tags/category → publish)
3. Interactions: Test like/dislike, comments, nested replies
4. Search: Verify search by keyword, tag filter, category filter
5. Themes: Toggle light/dark mode and verify persistence
6. Database: Check SQLite tables for correct relationships

Claude has written up a plan and is ready to execute. Would you like to proceed?
> 1. Yes, clear context and auto-accept edits (shift+tab)
2. Yes, auto-accept edits
3. Yes, manually approve edits
4. Type here to tell Claude what to change

```

Figure 9.2: AI negotiation mode in claude

After several rounds of negotiation, you can get a plan for the implementation of the blog system, you can accept the plan and proceed with the development or keep changing it until it is satisfactory.

Now we use vscode to open the project folder, and create a new file named 'task.md', then copy the plan generated by AI into this file, and we can refer to this file during the development process.

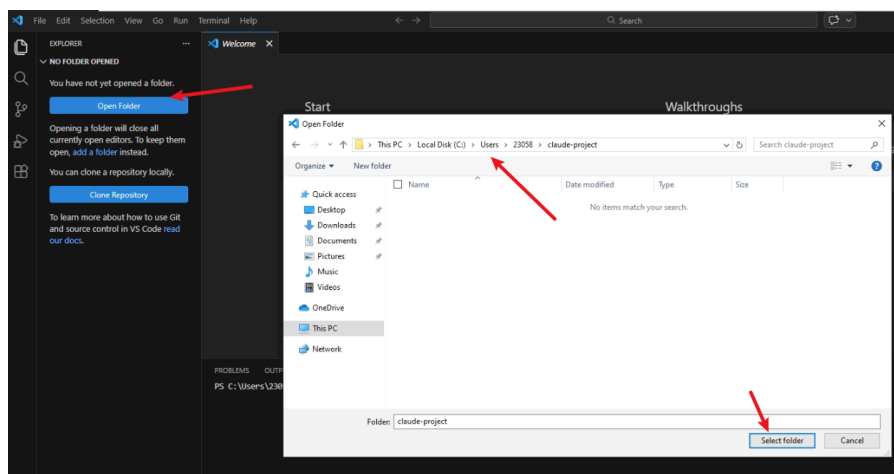


Figure 9.3: Open project folder in VS Code

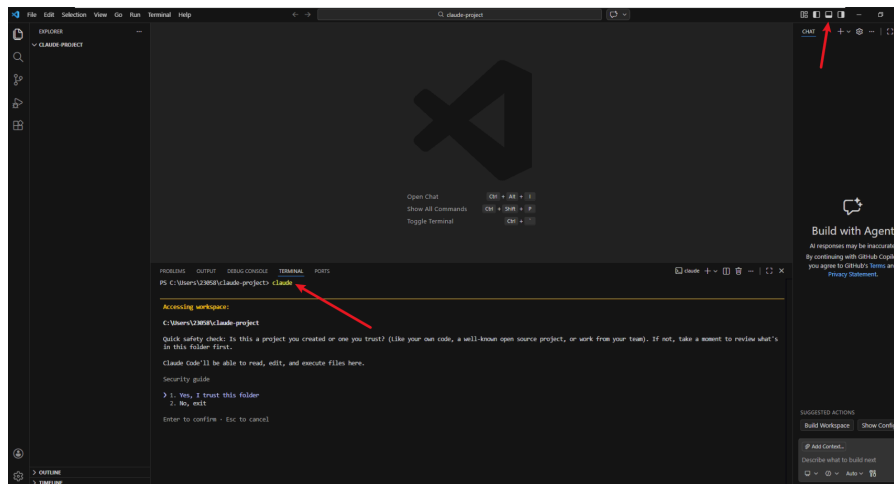


Figure 9.4: Use Claude in VS Code

9.2 Control Code Version and Save on Github

We firstly need to create an empty git repository on github, and copy the remote repository URL, then we will initialize a git repository in our project folder, and add the remote repository to our local git repository, then we can stage all the changes, commit the changes, and push the changes to the remote repository on github.

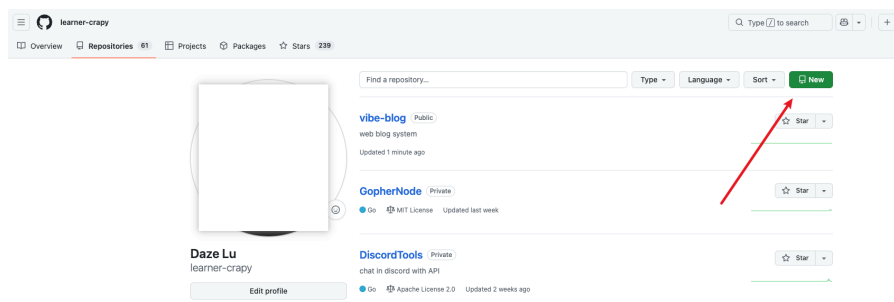


Figure 9.5: create a project on github

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner * **Repository name ***

 learner-crapy

✔ vibe-blog is available.

Great repository names are short and memorable. How about [special-octo-invention?](#)

Description

15 / 350 characters

2

Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Add README On ☒
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

Create repository

Figure 9.6: Create a new git repository on GitHub

 **Note** Please note that your github repo name should be same as your local project folder name.

We then need to initialize a git repository in our project folder. Follow these steps:

1. Initialize a git repository in your project folder by running:

```
git init
```

2. Create a new repository on GitHub and copy the remote repository URL.
3. Add the remote repository to your local git repository:

```
git remote add origin https://github.com/learner-crapy/vibe-blog.git
```

4. Stage all the changes:

```
git add .
```

5. Commit the changes:

```
git commit -m "initial commit"
```

6. Push the changes to the remote repository:

```
git push -u origin master
```

After several rounds of developments, you'll get a system.

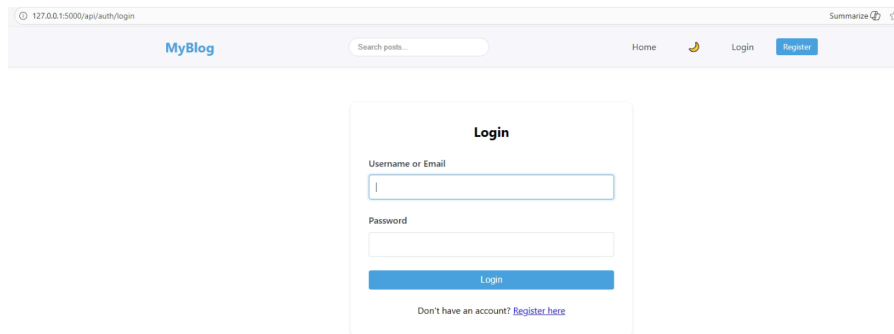


Figure 9.7: the blog system running on local machine

9.3 make your project publicly accessible

After you have implemented the blog system on your local machine, only your local machine can access it. If you want to make it publicly accessible, you need to deploy it to a cloud platform. In this class, we will use Cloudflare to deploy our blog system.

Follow these steps:

1. Download the Cloudflare tunnel client for Windows from:
<https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/downloads/>
2. Start the tunnel by running the following command in your terminal:

```
.\cloudflared.exe tunnel --url http://localhost:5000
```

This will create a public URL for your local blog system that you can share with others.

Windows

Download and install `cloudflared` via [winget](#):

```
winget install --id Cloudflare.cloudflared
```

Alternatively, download the latest release directly:

Type	32-bit	64-bit
Executable	Download	Download

Note

Instances of `cloudflared` do not automatically update on Windows. You will need to perform manual updates.

Figure 9.8: download cloudflare tunnel client for windows

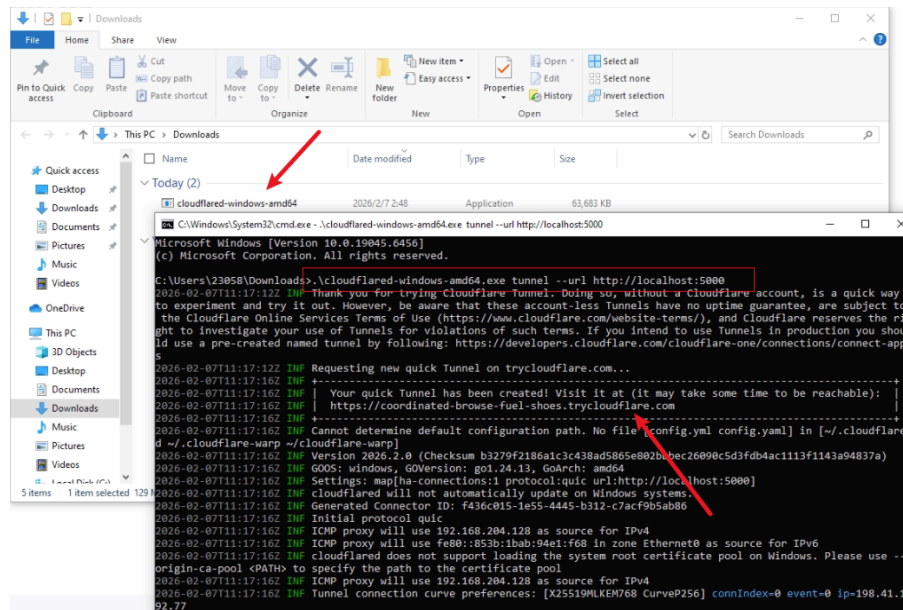


Figure 9.9: cloudflare tunnel client command to start the tunnel and get a public URL for your local blog system

Chapter 10 **vibe coding in data science**

Chapter 11 Mobile Development

Chapter 12 Desktop Development

Chapter 13 Game Development

Bibliography

- [1] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).